

Sri Lanka Institute of Information Technology



IT4100 – Software Quality Assurance

Year 4, Semester 2- 2026

SQA Test Cases Document

System URL– pixelssuite.com

Table of Contents

Kulasekara K S T	3
Manual Testing Documentation	3
Test Cases	3
Screenshots of manual test executions	5
Automated Testing Documentation	12
Automated test cases with scripts	12
Test execution results with screenshots	36
Defects found	53

Kulasekara K S T

Manual Testing Documentation

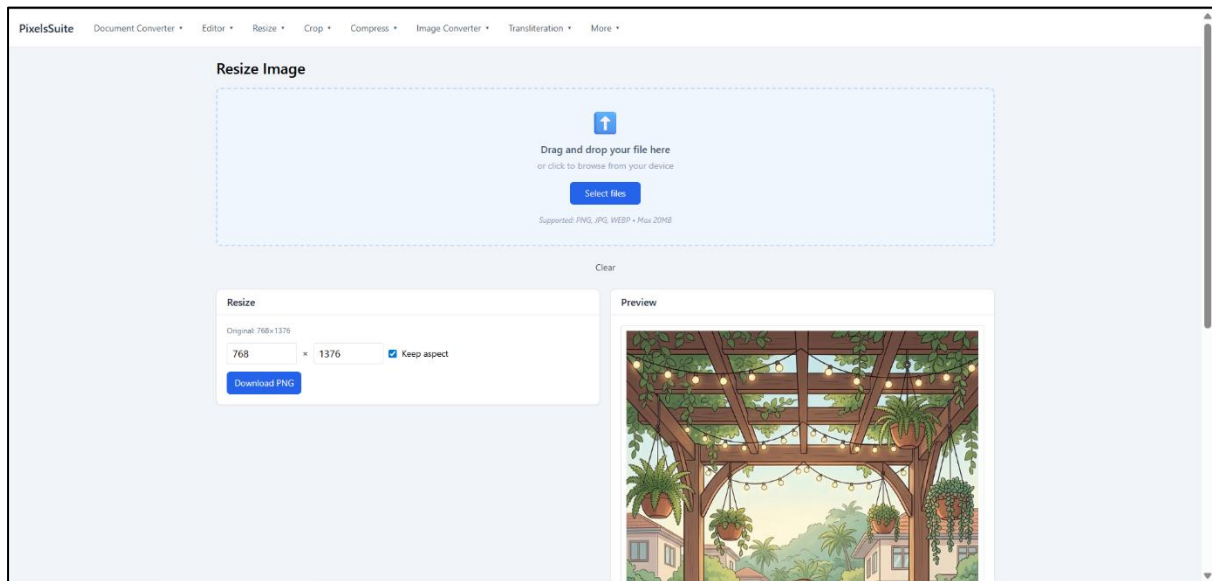
Test Cases

Test Case ID	Test Scenario	Test Steps	Expected Results	Actual Results	Status
TC_001	Upload a valid image file.	1. Open the 'Resize Image' page. 2. Click the 'select file' button. 3. Select a JPG/PNG/WEBP file.	The image should upload successfully	Image uploaded correctly	Pass
TC_002	Drag and drop a valid image file	1. Open the 'Resize Image' page. 2. Drag and drop a valid image file.	The image should upload successfully	Image uploaded correctly	Pass
TC_003	Upload an unsupported file	1. Click the 'select file' button. 2. Select unsupported file (e.g.: .txt, PDF)	The system should show an error message	The system doesn't allow uploading different files	Pass
TC_004	Drag and drop an unsupported image file	1. Open the 'Resize Image' page. 2. Drag and drop an unsupported image file (e.g., .txt, .PDF).	The system should show an error message	The system didn't show anything and didn't upload the file	Pass
TC_005	Type different sizes in the text box	1. Open the 'Resize Image' page. 2. Enter size values in the text box.	Image resizes according to the given size	Image resizes correctly and previews correctly	Pass
TC_006	Click the 'Download PNG' button.	1. Open the 'Resize Image' page. 2. Enter suitable size values. 3. Click the 'Download PNG' button.	Successfully downloaded the image.	Image downloaded successfully.	Pass
TC_007	Upload multiple valid image files.	1. Click the 'Bulk Resize' button in the drop-down. 2. Click the 'select file' button. 3. Select	The image should upload successfully	Image uploaded correctly	Pass

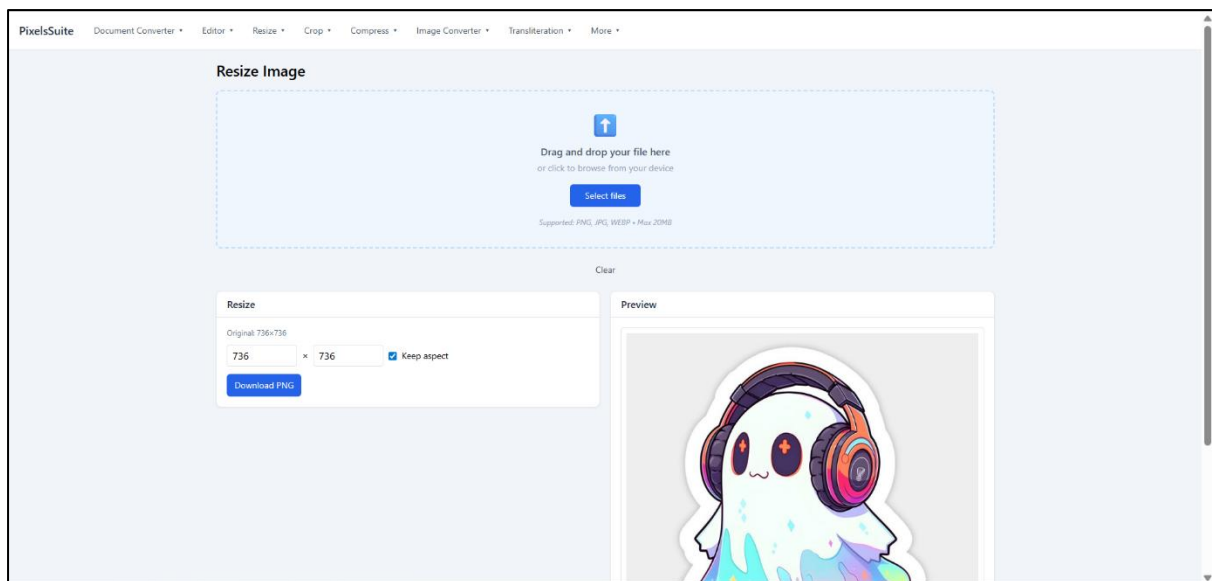
		JPG/PNG/WEBP multiple files.			
TC_008	Upload multiple unsupported files.	1. Click the 'select files' button. 2. Select unsupported file (e.g.: .txt, PDF)	The system should show an error message	Unsupported file uploaded correctly	Fail
TC_009	Click the 'Process & Download' button.	1. Open the 'Bulk Resize' page. 2. Enter suitable size values. 3. Click the 'Process & Download' button.	Successfully download the images.	Image downloaded successfully.	Pass
TC_010	Upload a valid image to the 'Image Enlarger' section	1. Click the 'Image Enlarger' button in the drop-down. 2. Click the 'select file' button. 3. Select a JPG/PNG/WEBP file.	The image should upload successfully	Image uploaded correctly	Pass
TC_011	Upload an unsupported file to the 'Image Enlarger' section.	1. Click the 'select files' button. 2. Select an unsupported file (e.g., .txt, PDF).	The system should show an error message	The system didn't show anything and didn't upload the file	Pass
TC_012	Click the 'Download PNG' button.	1. Open the 'Image Enlarger' page. 2. Give suitable scale values. 3. Click the 'Download PNG' button.	Successfully downloaded the image.	Successfully downloaded the image.	Pass
TC_013	Upload a large file (File size – 21MB)	1. Open the 'Resize Image' page. 2. Click the 'select file' button. 3. Select a large file.	The system should show an error message	Image uploaded successfully	Fail

Screenshots of manual test executions

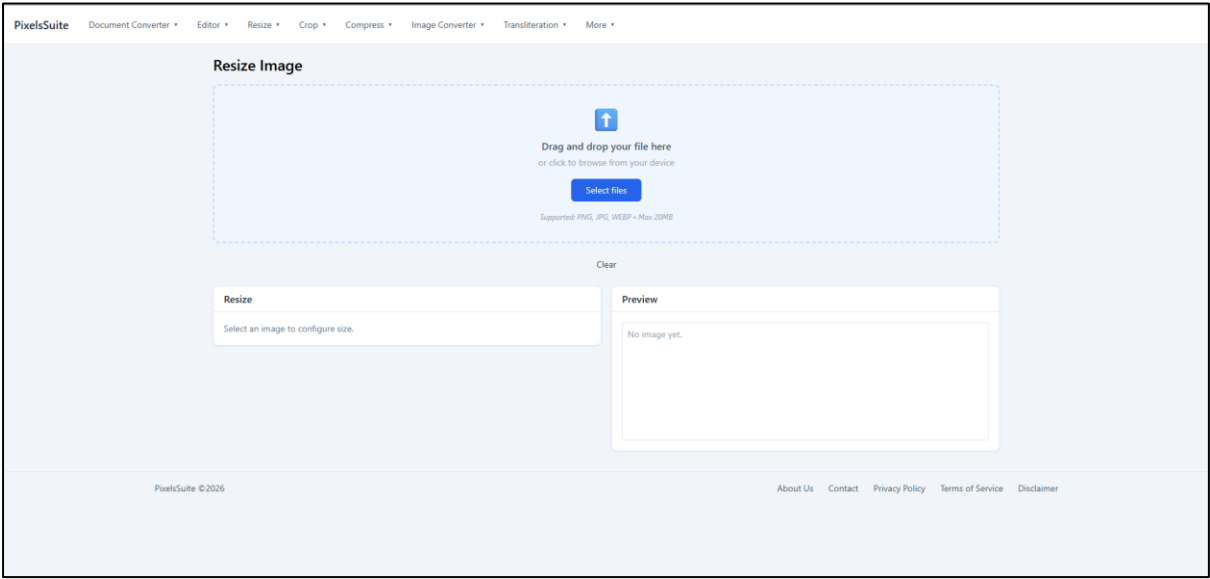
TC_001



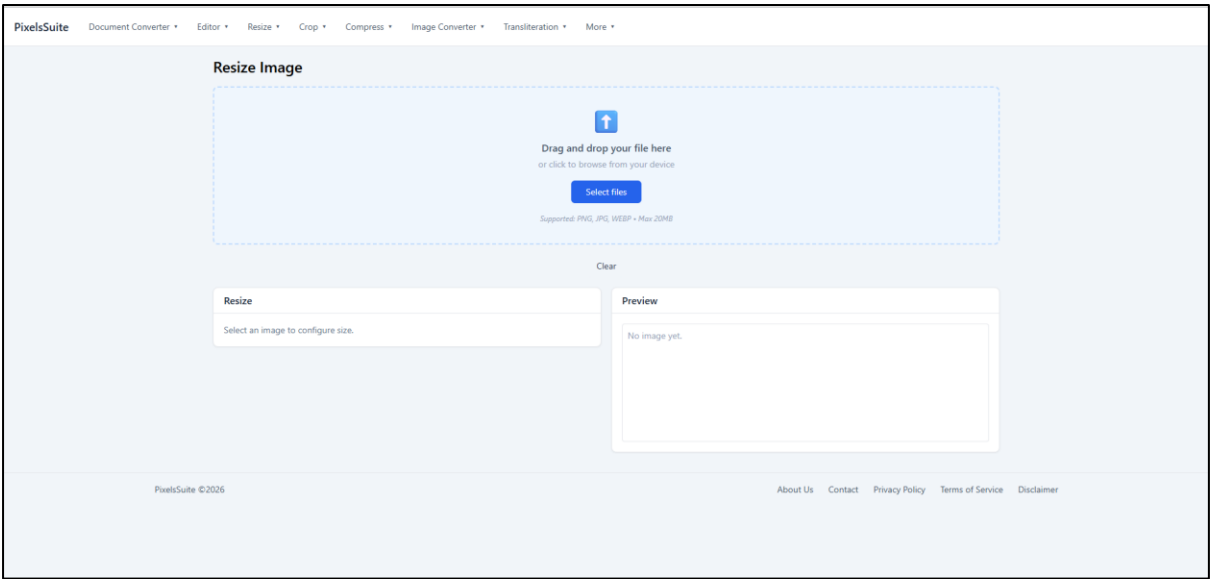
TC_002



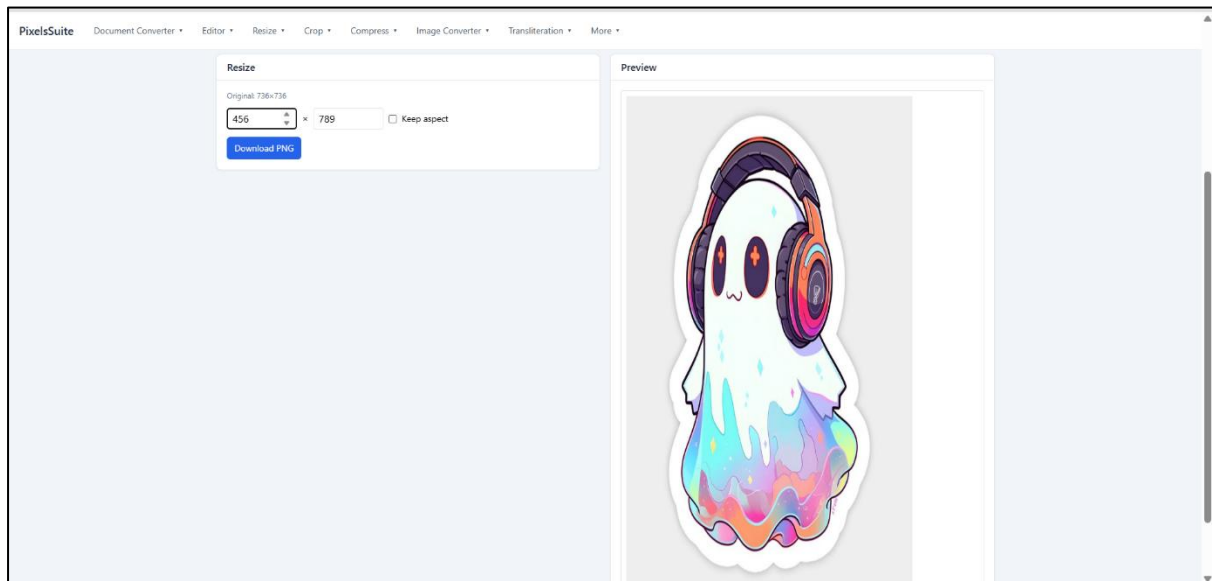
TC_003



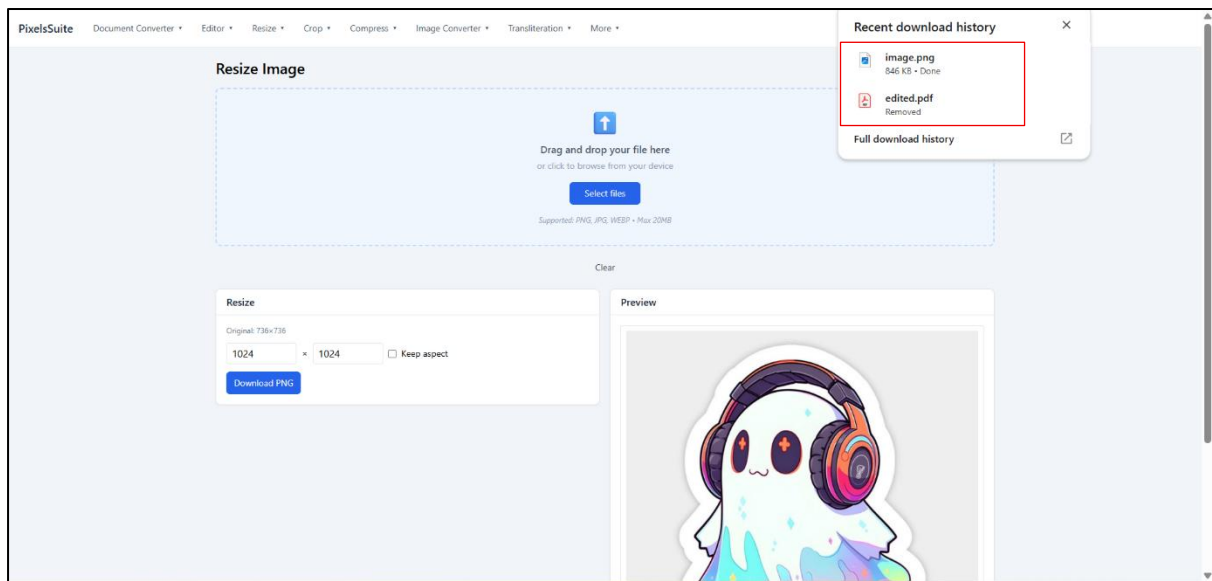
TC_004



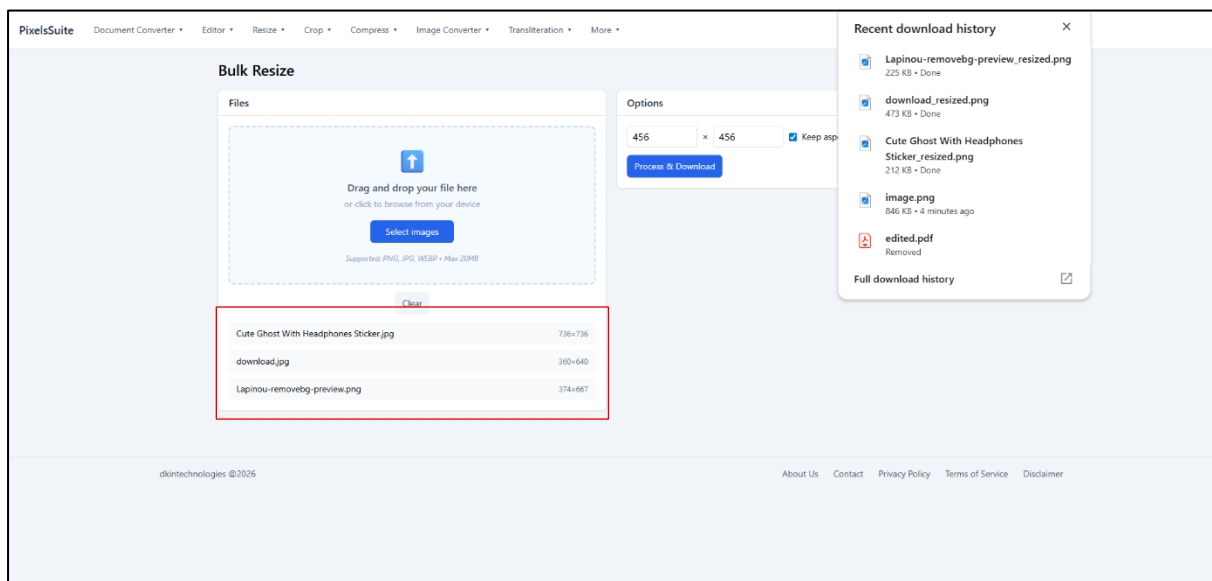
TC_005



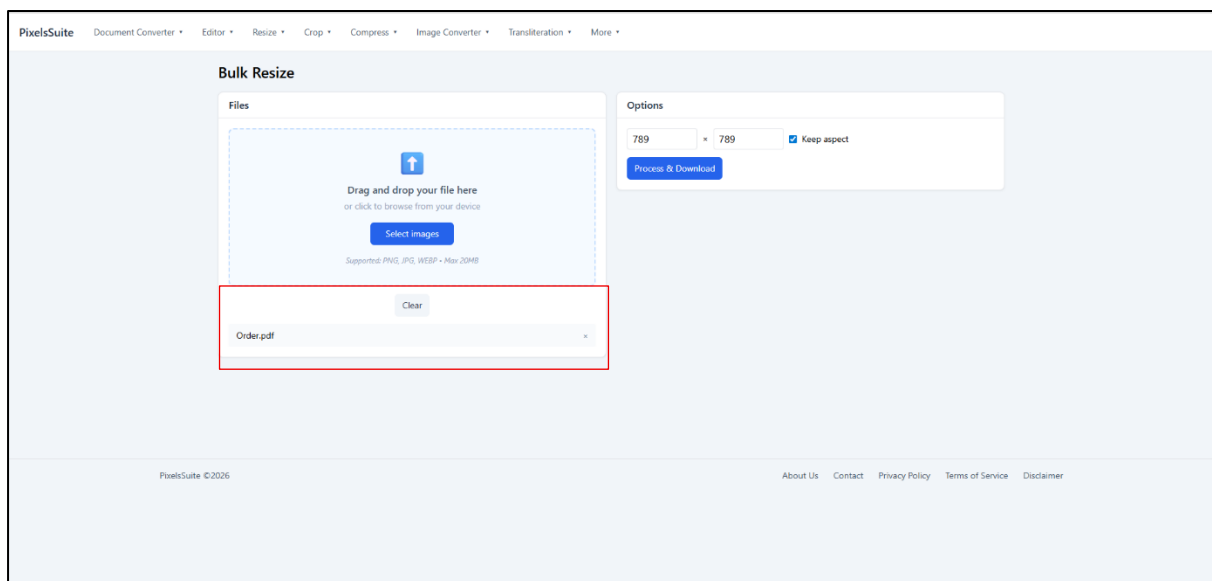
TC_006



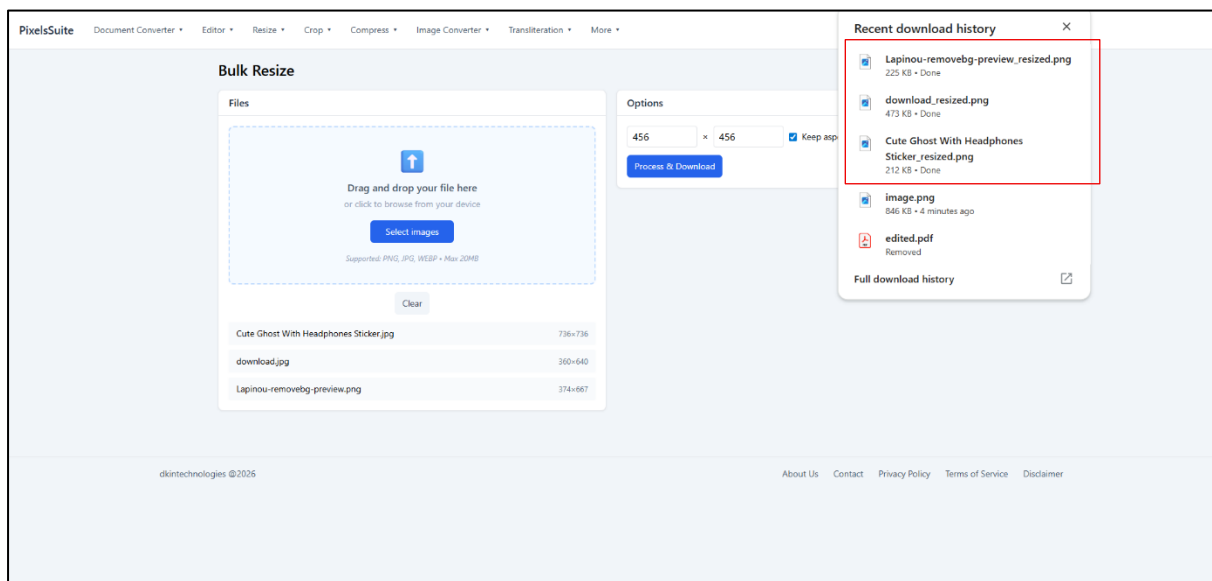
TC_007



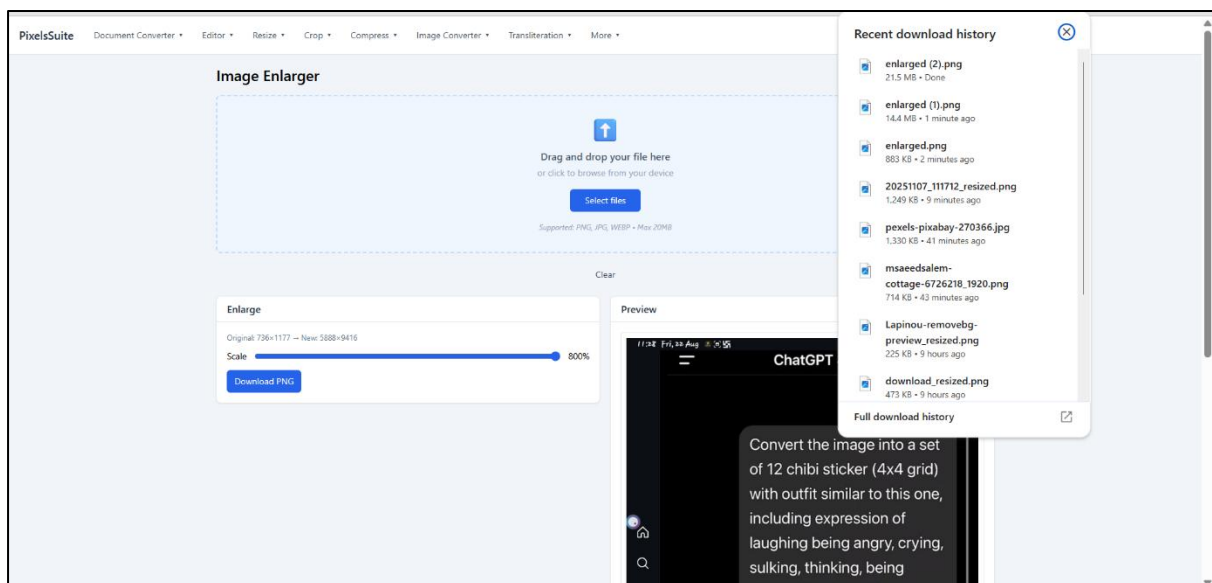
TC_008



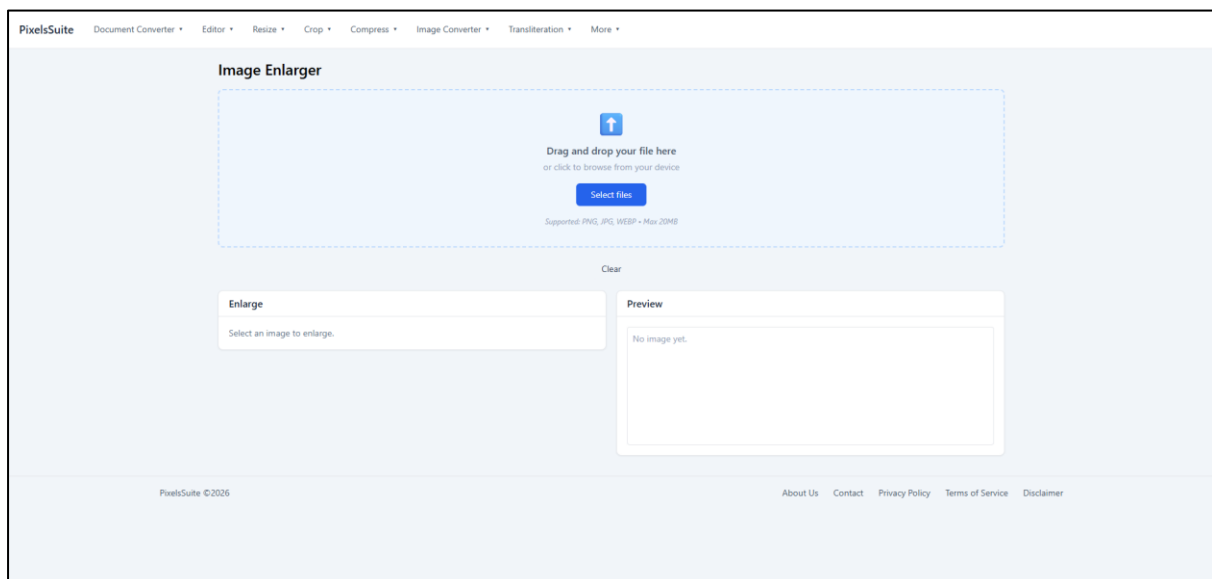
TC_009



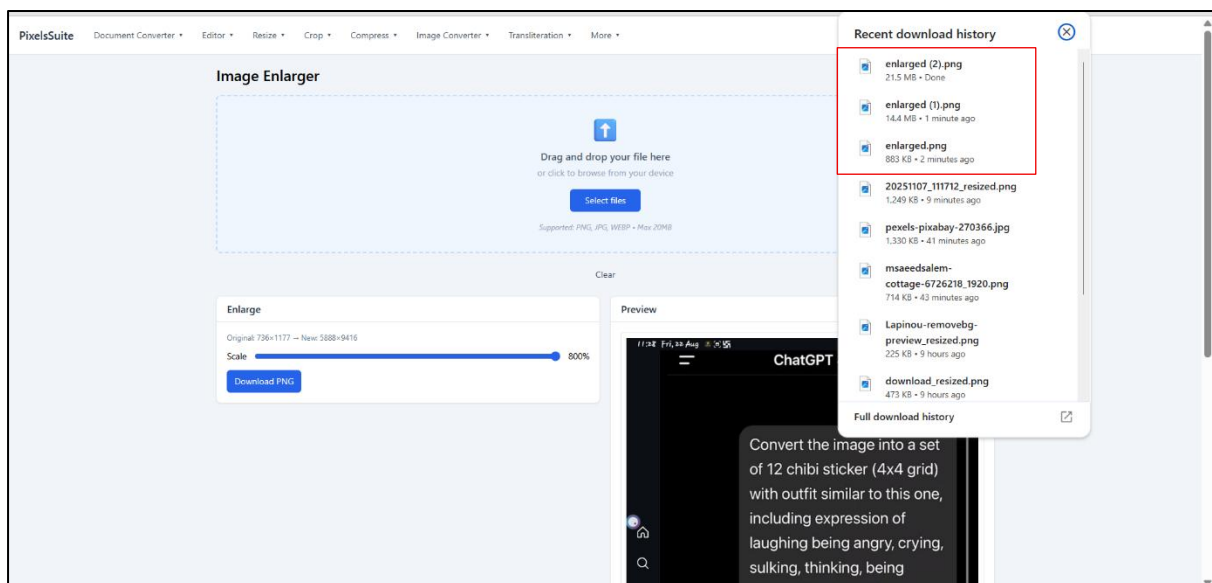
TC_010

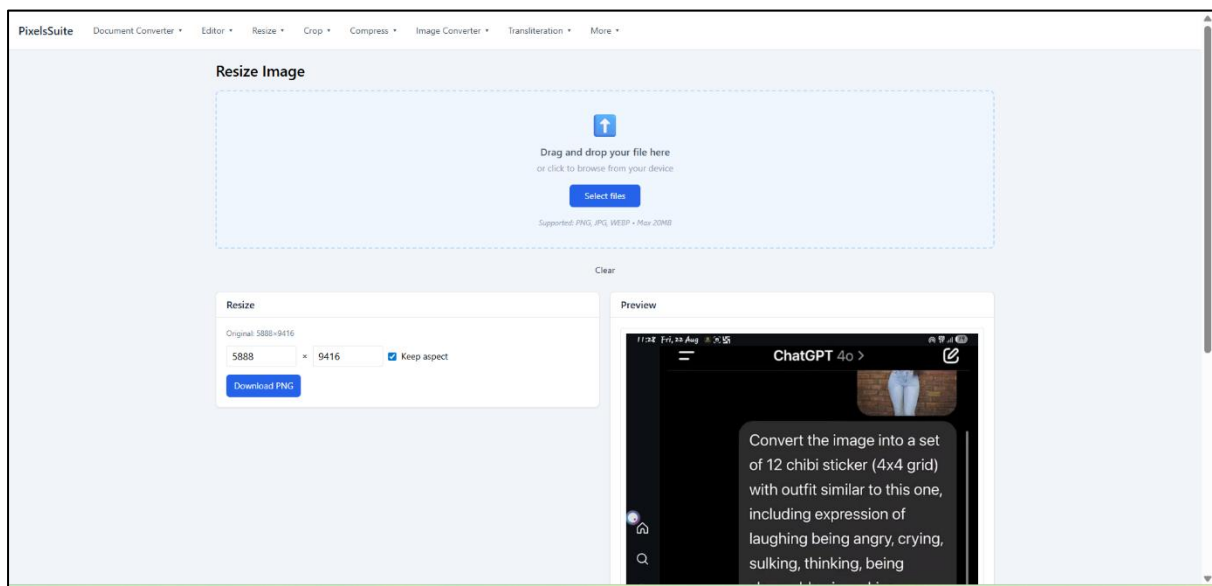


TC_011



TC_012





Automated Testing Documentation

Automated test cases with scripts

```
JS example.spec.js X
tests > JS example.spec.js > test('Create PDF without uploading file') callback
1  const { test, expect } = require('@playwright/test');
2
3  // Base URL
4  const URL = 'https://www.pixelssuite.com/';
5
6  // 1. Homepage Load Test
7  test('Homepage loads correctly', async ({ page }) => {
8    await page.goto(URL);
9    await expect(page).toHaveTitle(/.+/);
10   await expect(page.locator('body')).toBeVisible();
11  });
12
13  // 2. Dropdown Open Test
14  test('Document Converter dropdown opens', async ({ page }) => {
15    await page.goto(URL);
16
17    await page.getByText('Document Converter').hover();
18
19    await expect(page.getByText('Image → PDF')).toBeVisible();
20  });
21
22  // 3. Dropdown Navigation Test
23  test('Navigate to Image → PDF page', async ({ page }) => {
24    await page.goto(URL);
25
26    await page.getByText('Document Converter').hover();
27    await page.getByText('Image → PDF').click();
28
29    await page.waitForLoadState('load');
30
31    await expect(page).not.toHaveURL(URL);
32  });
33
```

```

34 // 4. Test Multiple Dropdown Links
35 test('Test multiple document converter links', async ({ page }) => {
36     await page.goto(URL);
37
38     const options = ['Image → PDF', 'PDF → Word', 'Word → PDF'];
39
40     for (let option of options) {
41         await page.getByText('Document Converter').hover();
42         await page.getByText(option).click();
43
44         await page.waitForLoadState('load');
45
46         console.log(`${option} page opened`);
47
48         await page.goto(URL);
49     }
50 });
51
52 // 5. Resize Menu Navigation
53 test('Resize menu navigation', async ({ page }) => {
54     await page.goto(URL);
55
56     await page.getByText('Resize').hover();
57     await page.getByText('Image Enlarger').click();
58
59     await page.waitForLoadState('load');
60
61     await expect(page).not.toHaveURL(URL);
62 });
63
64 // 6. Crop Menu Navigation
65 test('Crop menu navigation', async ({ page }) => {
66     await page.goto(URL);
67
68     await page.getByText('Crop').hover();
69     await page.getByText('To PNG').click();
70
71     await page.waitForLoadState('load');
72
73     await expect(page).not.toHaveURL(URL);
74 });

```

```

76 // 7. Compress Menu Navigation
77 test('Compress menu navigation', async ({ page }) => {
78     await page.goto(URL);
79
80     await page.getByText('Compress').hover();
81     await page.getByText('Compress Image').click();
82
83     await page.waitForLoadState('load');
84
85     await expect(page).not.toHaveURL(URL);
86 });
87
88 // 8. Input Field Test (textarea)
89 test('Text input works correctly', async ({ page }) => {
90     await page.goto(URL);
91
92     const input = page.locator('textarea').first();
93
94     if (await input.count() > 0) {
95         await input.fill('Hello Testing');
96         await expect(input).toHaveValue('Hello Testing');
97     } else {
98         console.log('No textarea found on homepage');
99     }
100 });
101
102 // 9. Button Click Test
103 test('Button click works', async ({ page }) => {
104     await page.goto(URL);
105
106     const button = page.getByRole('button').first();
107
108     if (await button.count() > 0) {
109         await button.click();
110         await page.waitForTimeout(2000);
111     } else {
112         console.log('No button found');
113     }
114 });

```

```

116 // 10. Edge Case - Empty Input
117 test('Empty input validation', async ({ page }) => {
118     await page.goto(URL);
119
120     const button = page.getByRole('button').first();
121
122     if (await button.count() > 0) {
123         await button.click();
124         await page.waitForTimeout(2000);
125
126         console.log('Checked empty input behavior');
127     }
128 });
129
130 // 11. Page Content Verification
131 test('Main content visible', async ({ page }) => {
132     await page.goto(URL);
133
134     await expect(page.getByText('Free Online Tools')).toBeVisible();
135 });
136
137 // 12. Screenshot Test (for report)
138 test('Take homepage screenshot', async ({ page }) => {
139     await page.goto(URL);
140
141     await page.screenshot({ path: 'homepage.png', fullPage: true });
142 });
143

```

```

144 // =====
145 // NEW TESTS (Image → PDF PAGE)
146 // =====
147
148 // 12. Open Image → PDF Page
149 test('Open Image to PDF page', async ({ page }) => {
150   await page.goto('https://www.pixelssuite.com/image-to-pdf');
151
152   await expect(page.getByText('Image → PDF')).toBeVisible();
153 });
154
155 // 13. Check UI Elements
156 test('Check all UI elements', async ({ page }) => {
157   await page.goto('https://www.pixelssuite.com/image-to-pdf');
158
159   await expect(page.getByText('Select Images')).toBeVisible();
160   await expect(page.getByText('Create PDF')).toBeVisible();
161   await expect(page.getByText('A4')).toBeVisible();
162   await expect(page.getByText('Portrait')).toBeVisible();
163 });
164
165 // 14. Upload image
166 test('Upload image file', async ({ page }) => {
167   await page.goto('https://www.pixelssuite.com/image-to-pdf');
168
169   const fileInput = page.locator('input[type="file"]');
170
171   await fileInput.setInputFiles('tests/files/test.png');
172
173   // Wait for preview or selection
174   await page.waitForTimeout(2000);
175
176   console.log('Image uploaded');
177 });
178

```



```

179 // 15. Click options
180 test('Change settings options', async ({ page }) => {
181   await page.goto('https://www.pixelssuite.com/image-to-pdf');
182
183   await page.getByText('Letter').click();
184   await page.getByText('Landscape').click();
185   await page.getByText('Horizontal').click();
186   await page.getByText('Multiple').click();
187
188   console.log('Options changed');
189 });
190
191 // 16. Create PDF
192 test('Create PDF after upload', async ({ page }) => {
193   await page.goto('https://www.pixelssuite.com/image-to-pdf');
194
195   const fileInput = page.locator('input[type="file"]');
196   await fileInput.setInputFiles('tests/files/test.png');
197
198   await page.waitForTimeout(2000);
199
200   await page.getByText('Create PDF').click();
201
202   await page.waitForTimeout(3000);
203
204   console.log('Create PDF clicked');
205 });
206

```

```

207 // 17. Edge case - Document Converter
208 test('Create PDF without uploading file', async ({ page }) => {
209   await page.goto('https://www.pixelssuite.com/image-to-pdf');
210
211   await page.getByText('Create PDF').click();
212
213   await page.waitForTimeout(2000);
214
215   // Expect some validation (may fail = BUG)
216   console.log('Checked empty upload case');
217 });
218
219 test('Screenshot Image to PDF page', async ({ page }) => {
220   await page.goto('https://www.pixelssuite.com/image-to-pdf');
221
222   await page.screenshot({ path: 'image-to-pdf.png', fullPage: true });
223 });
224

```

```

225 // =====
226 // WORD → PDF PAGE TESTS
227 // =====
228
229 test('Open Word to PDF page', async ({ page }) => {
230   await page.goto('https://www.pixelssuite.com/word-to-pdf');
231   await expect(page.getByText('Word → PDF')).toBeVisible();
232 });
233
234 test('Word to PDF UI elements visible', async ({ page }) => {
235   await page.goto('https://www.pixelssuite.com/word-to-pdf');
236
237   await expect(page.getByText('Select Word')).toBeVisible();
238   await expect(page.getByText('Drag and drop your file here')).toBeVisible();
239 });
240
241 test('Upload Word file', async ({ page }) => {
242   await page.goto('https://www.pixelssuite.com/word-to-pdf');
243
244   const fileInput = page.locator('input[type="file"]');
245   await fileInput.setInputFiles('tests/files/test.docx');
246
247   await page.waitForTimeout(2000);
248 });
249
250 test('Word to PDF without file (edge case)', async ({ page }) => {
251   await page.goto('https://www.pixelssuite.com/word-to-pdf');
252
253   // Try clicking upload area or button
254   await page.getByText('Select Word').click();
255
256   await page.waitForTimeout(2000);
257 });
258
259 test('Word to PDF screenshot', async ({ page }) => {
260   await page.goto('https://www.pixelssuite.com/word-to-pdf');
261
262   await page.screenshot({ path: 'word-to-pdf.png', fullPage: true });
263 });
264

```

```

266 // =====
267 // PDF → WORD PAGE TESTS
268 // =====
269
270 test('Open PDF to Word page', async ({ page }) => {
271   await page.goto('https://www.pixelssuite.com/pdf-to-word');
272   await expect(page.getByText('PDF → Word')).toBeVisible();
273 });
274
275 test('PDF to Word UI elements visible', async ({ page }) => {
276   await page.goto('https://www.pixelssuite.com/pdf-to-word');
277
278   await expect(page.getByText('Select PDF')).toBeVisible();
279   await expect(page.getByText('Drag and drop your file here')).toBeVisible();
280 });
281
282 test('Upload PDF file', async ({ page }) => {
283   await page.goto('https://www.pixelssuite.com/pdf-to-word');
284
285   const fileInput = page.locator('input[type="file"]');
286   await fileInput.setInputFiles('tests/files/test.pdf');
287
288   await page.waitForTimeout(2000);
289 });
290
291 test('PDF to Word without file (edge case)', async ({ page }) => {
292   await page.goto('https://www.pixelssuite.com/pdf-to-word');
293
294   await page.getByText('Select PDF').click();
295
296   await page.waitForTimeout(2000);
297 });
298
299 test('PDF to Word screenshot', async ({ page }) => {
300   await page.goto('https://www.pixelssuite.com/pdf-to-word');
301
302   await page.screenshot({ path: 'pdf-to-word.png', fullPage: true });
303 });

```

```

305 // =====
306 // PDF EDITOR PAGE TESTS
307 // =====
308
309 // 1. Open PDF Editor page and verify heading
310 test('Open PDF Editor page', async ({ page }) => {
311     await page.goto('https://www.pixelssuite.com/pdf-editor');
312
313     // Verify page title/heading is visible
314     await expect(page.getByText('PDF Editor')).toBeVisible();
315 });
316
317
318 // 2. Verify main UI elements are visible
319 test('PDF Editor UI elements visible', async ({ page }) => {
320     await page.goto('https://www.pixelssuite.com/pdf-editor');
321
322     // Check file upload button
323     await expect(page.getByText('Choose File')).toBeVisible();
324
325     // Check toolbar elements
326     await expect(page.locator('text=Toolbar')).toBeVisible();
327
328     // Check Download button
329     await expect(page.getByText('Download')).toBeVisible();
330
331     // Check page navigation controls
332     await expect(page.getByText('Prev')).toBeVisible();
333     await expect(page.getByText('Next')).toBeVisible();
334 });
335

```

```

337 // 3. Upload PDF file into editor
338 test('Upload PDF file into editor', async ({ page }) => {
339   await page.goto('https://www.pixelssuite.com/pdf-editor');
340
341   // Locate file input
342   const fileInput = page.locator('input[type="file"]');
343
344   // Upload a sample PDF file
345   await fileInput.setInputFiles('tests/files/test.pdf');
346
347   // Wait for file to load in editor
348   await page.waitForTimeout(3000);
349
350   console.log('PDF file uploaded');
351 });
352
353
354 // 4. Toolbar interaction (click tools)
355 test('Toolbar buttons interaction', async ({ page }) => {
356   await page.goto('https://www.pixelssuite.com/pdf-editor');
357
358   // Click text tool (T icon)
359   await page.locator('button:has-text("T")').click().catch(() => {});
360
361   // Click bold (B)
362   await page.locator('text=B').click().catch(() => {});
363
364   // Click alignment options (L, C, R)
365   await page.locator('text=L').click().catch(() => {});
366   await page.locator('text=C').click().catch(() => {});
367   await page.locator('text=R').click().catch(() => {});
368
369   console.log('Toolbar interactions tested');
370 });
371

```

```

373 // 5. Zoom control interaction
374 test('Zoom slider interaction', async ({ page }) => {
375     await page.goto('https://www.pixelssuite.com/pdf-editor');
376
377     // Locate zoom slider and change value
378     const zoomSlider = page.locator('input[type="range"]');
379
380     if (await zoomSlider.count() > 0) {
381         await zoomSlider.fill('150');
382     }
383
384     await page.waitForTimeout(2000);
385
386     console.log('Zoom adjusted');
387 });
388
389
390 // 6. Page navigation buttons
391 test('Page navigation buttons', async ({ page }) => {
392     await page.goto('https://www.pixelssuite.com/pdf-editor');
393
394     // Click next and previous buttons
395     await page.getByText('Next').click().catch(() => {});
396     await page.getByText('Prev').click().catch(() => {});
397
398     await page.waitForTimeout(2000);
399
400     console.log('Page navigation tested');
401 });
402
403
404 // 7. Download button without uploading file (EDGE CASE)
405 test('Download without uploading PDF (edge case)', async ({ page }) => {
406     await page.goto('https://www.pixelssuite.com/pdf-editor');
407
408     // Click download without uploading file
409     await page.getByText('Download').click();
410
411     await page.waitForTimeout(2000);
412
413     // Expect validation or no response (possible defect)
414     console.log('Checked download without file');
415 });
416
417
418 // 8. Screenshot for report
419 test('PDF Editor screenshot', async ({ page }) => {
420     await page.goto('https://www.pixelssuite.com/pdf-editor');
421
422     // Capture full page screenshot
423     await page.screenshot({ path: 'pdf-editor.png', fullPage: true });
424 });
425

```

```

426 // =====
427 // RESIZE IMAGE PAGE TESTS
428 // =====
429
430 // 1. Open Resize Image page
431 test('Open Resize Image page', async ({ page }) => {
432   await page.goto('https://www.pixelssuite.com/resize');
433
434   // Verify page title
435   await expect(page.getByText('Resize Image')).toBeVisible();
436 });
437
438
439 // 2. Check UI elements
440 test('Resize Image UI elements', async ({ page }) => {
441   await page.goto('https://www.pixelssuite.com/resize');
442
443   // Check upload button and preview section
444   await expect(page.getByText('Select files')).toBeVisible();
445   await expect(page.getByText('Preview')).toBeVisible();
446 });
447
448
449 // 3. Upload image
450 test('Upload image in Resize page', async ({ page }) => {
451   await page.goto('https://www.pixelssuite.com/resize');
452
453   // Locate file input
454   const fileInput = page.locator('input[type="file"]');
455
456   // Upload test image
457   await fileInput.setInputFiles('tests/files/test.png');
458
459   await page.waitForTimeout(2000);
460 });
461
462
463 // 4. Screenshot
464 test('Resize Image screenshot', async ({ page }) => {
465   await page.goto('https://www.pixelssuite.com/resize');
466
467   await page.screenshot({ path: 'resize.png', fullPage: true });
468 });

```

```

471 // =====
472 // BULK RESIZE PAGE TESTS
473 // =====
474
475 // 5. Open Bulk Resize page
476 test('Open Bulk Resize page', async ({ page }) => {
477     await page.goto('https://www.pixelssuite.com/batch-resize');
478
479     await expect(page.getByText('Bulk Resize')).toBeVisible();
480 });
481
482
483 // 6. Check UI elements
484 test('Bulk Resize UI elements', async ({ page }) => {
485     await page.goto('https://www.pixelssuite.com/batch-resize');
486
487     // Verify input fields and button
488     await expect(page.getByPlaceholder('Width')).toBeVisible();
489     await expect(page.getByPlaceholder('Height')).toBeVisible();
490     await expect(page.getByText('Process & Download')).toBeVisible();
491 });
492
493
494 // 7. Upload multiple images
495 test('Upload multiple images', async ({ page }) => {
496     await page.goto('https://www.pixelssuite.com/batch-resize');
497
498     const fileInput = page.locator('input[type="file"]');
499
500     // Upload multiple files
501     await fileInput.setInputFiles([
502         'tests/files/test.png',
503         'tests/files/test2.png'
504     ]);
505
506     await page.waitForTimeout(2000);
507 });
508

```



```

510 // 8. Enter dimensions and process
511 test('Enter dimensions and process images', async ({ page }) => {
512     await page.goto('https://www.pixelssuite.com/batch-resize');
513
514     const fileInput = page.locator('input[type="file"]');
515     await fileInput.setInputFiles('tests/files/test.png');
516
517     // Enter width & height
518     await page.getByPlaceholder('Width').fill('500');
519     await page.getByPlaceholder('Height').fill('500');
520
521     // Click process button
522     await page.getByText('Process & Download').click();
523
524     await page.waitForTimeout(3000);
525 });
526
527
528 // 9. Edge case (no upload)
529 test('Bulk Resize without upload (edge case)', async ({ page }) => {
530     await page.goto('https://www.pixelssuite.com/batch-resize');
531
532     await page.getByText('Process & Download').click();
533
534     await page.waitForTimeout(2000);
535 });
536

```

```

538 // =====
539 // IMAGE ENLARGER PAGE TESTS
540 // =====
541
542 // 10. Open Image Enlarger page
543 test('Open Image Enlarger page', async ({ page }) => {
544   await page.goto('https://www.pixelssuite.com/image-enlarger');
545
546   await expect(page.getByText('Image Enlarger')).toBeVisible();
547 });
548
549
550 // 11. Check UI elements
551 test('Image Enlarger UI elements', async ({ page }) => {
552   await page.goto('https://www.pixelssuite.com/image-enlarger');
553
554   await expect(page.getByText('Select files')).toBeVisible();
555   await expect(page.getByText('Preview')).toBeVisible();
556 });
557
558
559 // 12. Upload image
560 test('Upload image in Image Enlarger', async ({ page }) => {
561   await page.goto('https://www.pixelssuite.com/image-enlarger');
562
563   const fileInput = page.locator('input[type="file"]');
564
565   await fileInput.setInputFiles('tests/files/test.png');
566
567   await page.waitForTimeout(2000);
568 });
569
570
571 // 13. Screenshot
572 test('Image Enlarger screenshot', async ({ page }) => {
573   await page.goto('https://www.pixelssuite.com/image-enlarger');
574
575   await page.screenshot({ path: 'image-enlarger.png', fullPage: true });
576 });
577

```

```

578 // =====
579 // CROP PNG PAGE TESTS
580 // =====
581
582 // 1. Open Crop PNG page
583 test('Open Crop PNG page', async ({ page }) => {
584   await page.goto('https://www.pixelssuite.com/crop-png');
585
586   // Verify page title
587   await expect(page.getByText('Crop PNG')).toBeVisible();
588 });
589
590
591 // 2. Check UI elements
592 test('Crop PNG UI elements', async ({ page }) => {
593   await page.goto('https://www.pixelssuite.com/crop-png');
594
595   // Check upload area and preview section
596   await expect(page.getByText('Select files')).toBeVisible();
597   await expect(page.getByText('Preview')).toBeVisible();
598 });
599
600
601 // 3. Upload PNG image
602 test('Upload PNG image for cropping', async ({ page }) => {
603   await page.goto('https://www.pixelssuite.com/crop-png');
604
605   const fileInput = page.locator('input[type="file"]');
606
607   // Upload PNG file
608   await fileInput.setInputFiles('tests/files/test.png');
609
610   await page.waitForTimeout(2000);
611 });
612
613
614 // 4. Screenshot
615 test('Crop PNG screenshot', async ({ page }) => {
616   await page.goto('https://www.pixelssuite.com/crop-png');
617
618   await page.screenshot({ path: 'crop-png.png', fullPage: true });
619 });
620

```

```

622 // =====
623 // CROP JPG PAGE TESTS
624 // =====
625
626 // 5. Open Crop JPG page
627 test('Open Crop JPG page', async ({ page }) => {
628   await page.goto('https://www.pixelssuite.com/crop-jpg');
629
630   await expect(page.getByText('Crop JPG')).toBeVisible();
631 });
632
633
634 // 6. Check UI elements
635 test('Crop JPG UI elements', async ({ page }) => {
636   await page.goto('https://www.pixelssuite.com/crop-jpg');
637
638   await expect(page.getByText('Select files')).toBeVisible();
639   await expect(page.getByText('Preview')).toBeVisible();
640 });
641
642
643 // 7. Upload JPG image
644 test('Upload JPG image for cropping', async ({ page }) => {
645   await page.goto('https://www.pixelssuite.com/crop-jpg');
646
647   const fileInput = page.locator('input[type="file"]');
648
649   // Upload JPG file
650   await fileInput.setInputFiles('tests/files/test.jpg');
651
652   await page.waitForTimeout(2000);
653 });
654
655
656 // 8. Screenshot
657 test('Crop JPG screenshot', async ({ page }) => {
658   await page.goto('https://www.pixelssuite.com/crop-jpg');
659
660   await page.screenshot({ path: 'crop-jpg.png', fullPage: true });
661 });
662

```

```

664 // =====
665 // CROP WEBP PAGE TESTS
666 // =====
667
668 // 9. Open Crop WebP page
669 test('Open Crop WebP page', async ({ page }) => {
670   await page.goto('https://www.pixelssuite.com/crop-webp');
671
672   await expect(page.getByText('Crop WebP')).toBeVisible();
673 });
674
675
676 // 10. Check UI elements
677 test('Crop WebP UI elements', async ({ page }) => {
678   await page.goto('https://www.pixelssuite.com/crop-webp');
679
680   await expect(page.getByText('Select files')).toBeVisible();
681   await expect(page.getByText('Preview')).toBeVisible();
682 });
683
684
685 // 11. Upload WebP image
686 test('Upload WebP image for cropping', async ({ page }) => {
687   await page.goto('https://www.pixelssuite.com/crop-webp');
688
689   const fileInput = page.locator('input[type="file"]');
690
691   // Upload WebP file
692   await fileInput.setInputFiles('tests/files/test.webp');
693
694   await page.waitForTimeout(2000);
695 });
696
697
698 // 12. Screenshot
699 test('Crop WebP screenshot', async ({ page }) => {
700   await page.goto('https://www.pixelssuite.com/crop-webp');
701
702   await page.screenshot({ path: 'crop-webp.png', fullPage: true });
703 });
704

```

```

705 // =====
706 // COMPRESS IMAGE PAGE TESTS
707 // =====
708
709 // 1. Open Compress Image page
710 test('Open Compress Image page', async ({ page }) => {
711   await page.goto('https://www.pixelssuite.com/compress');
712
713   // Verify page title
714   await expect(page.getByText('Compress Image')).toBeVisible();
715 });
716
717
718 // 2. Check UI elements
719 test('Compress Image UI elements', async ({ page }) => {
720   await page.goto('https://www.pixelssuite.com/compress');
721
722   // Check upload button and preview section
723   await expect(page.getByText('Select files')).toBeVisible();
724   await expect(page.getByText('Preview')).toBeVisible();
725 });
726
727
728 // 3. Upload image
729 test('Upload image for compression', async ({ page }) => {
730   await page.goto('https://www.pixelssuite.com/compress');
731
732   const fileInput = page.locator('input[type="file"]');
733
734   // Upload test image (png or jpg)
735   await fileInput.setInputFiles('tests/files/test.png');
736
737   await page.waitForTimeout(2000);
738 });
739
740
741 // 4. Screenshot
742 test('Compress Image screenshot', async ({ page }) => {
743   await page.goto('https://www.pixelssuite.com/compress');
744
745   await page.screenshot({ path: 'compress.png', fullPage: true });
746 });
747

```

```

749 // =====
750 // PNG COMPRESSOR PAGE TESTS
751 // =====
752
753 // 5. Open PNG Compressor page
754 test('Open PNG Compressor page', async ({ page }) => {
755     await page.goto('https://www.pixelssuite.com/compress-png');
756
757     await expect(page.getByText('PNG Compressor')).toBeVisible();
758 });
759
760
761 // 6. Check UI elements
762 test('PNG Compressor UI elements', async ({ page }) => {
763     await page.goto('https://www.pixelssuite.com/compress-png');
764
765     await expect(page.getByText('Select files')).toBeVisible();
766     await expect(page.getByText('Preview')).toBeVisible();
767 });
768
769
770 // 7. Upload PNG image
771 test('Upload PNG for compression', async ({ page }) => {
772     await page.goto('https://www.pixelssuite.com/compress-png');
773
774     const fileInput = page.locator('input[type="file"]');
775
776     // Upload PNG file
777     await fileInput.setInputFiles('tests/files/test.png');
778
779     await page.waitForTimeout(2000);
780 });
781
782
783 // 8. Screenshot
784 test('PNG Compressor screenshot', async ({ page }) => {
785     await page.goto('https://www.pixelssuite.com/compress-png');
786
787     await page.screenshot({ path: 'compress-png.png', fullPage: true });
788 });
789

```

```

791 // =====
792 // GIF COMPRESSOR PAGE TESTS
793 // =====
794
795 // 9. Open GIF Compressor page
796 test('Open GIF Compressor page', async ({ page }) => {
797   await page.goto('https://www.pixelssuite.com/compress-gif');
798
799   await expect(page.getByText('GIF Compressor')).toBeVisible();
800 });
801
802
803 // 10. Check UI elements
804 test('GIF Compressor UI elements', async ({ page }) => {
805   await page.goto('https://www.pixelssuite.com/compress-gif');
806
807   await expect(page.getByText('Select GIF')).toBeVisible();
808   await expect(page.getByText('Preview')).toBeVisible();
809 });
810
811
812 // 11. Upload GIF file
813 test('Upload GIF for compression', async ({ page }) => {
814   await page.goto('https://www.pixelssuite.com/compress-gif');
815
816   const fileInput = page.locator('input[type="file"]');
817
818   // Upload GIF file
819   await fileInput.setInputFiles('tests/files/test.gif');
820
821   await page.waitForTimeout(2000);
822 });
823
824
825 // 12. Screenshot
826 test('GIF Compressor screenshot', async ({ page }) => {
827   await page.goto('https://www.pixelssuite.com/compress-gif');
828
829   await page.screenshot({ path: 'compress-gif.png', fullPage: true });
830 });
831

```



```

832 // =====
833 // IMAGE → PNG PAGE TESTS
834 // =====
835
836 // 1. Open Image to PNG page
837 test('Open Image to PNG page', async ({ page }) => {
838   await page.goto('https://www.pixelssuite.com/to-png');
839
840   // Verify page title
841   await expect(page.getByText('Convert Image')).toBeVisible();
842 });
843
844
845 // 2. Check UI elements
846 test('Image to PNG UI elements', async ({ page }) => {
847   await page.goto('https://www.pixelssuite.com/to-png');
848
849   // Check upload and preview sections
850   await expect(page.getByText('Select files')).toBeVisible();
851   await expect(page.getByText('Preview')).toBeVisible();
852 });
853
854
855 // 3. Upload image and convert
856 test('Upload image for PNG conversion', async ({ page }) => {
857   await page.goto('https://www.pixelssuite.com/to-png');
858
859   const fileInput = page.locator('input[type="file"]');
860
861   // Upload test image
862   await fileInput.setInputFiles('tests/files/test.jpg');
863
864   await page.waitForTimeout(2000);
865 });
866
867
868 // 4. Screenshot
869 test('Image to PNG screenshot', async ({ page }) => {
870   await page.goto('https://www.pixelssuite.com/to-png');
871
872   await page.screenshot({ path: 'to-png.png', fullPage: true });
873 });
874

```

```

876 // =====
877 // IMAGE → JPG PAGE TESTS
878 // =====
879
880 // 5. Open Image to JPG page
881 test('Open Image to JPG page', async ({ page }) => {
882   await page.goto('https://www.pixelssuite.com/to-jpg');
883
884   await expect(page.getByText('Convert Image')).toBeVisible();
885 });
886
887
888 // 6. Check UI elements
889 test('Image to JPG UI elements', async ({ page }) => {
890   await page.goto('https://www.pixelssuite.com/to-jpg');
891
892   await expect(page.getByText('Select files')).toBeVisible();
893   await expect(page.getByText('Preview')).toBeVisible();
894 });
895
896
897 // 7. Upload image and convert
898 test('Upload image for JPG conversion', async ({ page }) => {
899   await page.goto('https://www.pixelssuite.com/to-jpg');
900
901   const fileInput = page.locator('input[type="file"]');
902
903   // Upload PNG or WebP file
904   await fileInput.setInputFiles('tests/files/test.png');
905
906   await page.waitForTimeout(2000);
907 });
908
909
910 // 8. Screenshot
911 test('Image to JPG screenshot', async ({ page }) => {
912   await page.goto('https://www.pixelssuite.com/to-jpg');
913
914   await page.screenshot({ path: 'to-jpg.png', fullPage: true });
915 });
916

```

```

918 // =====
919 // IMAGE → WEBP PAGE TESTS
920 // =====
921
922 // 9. Open Image to WebP page
923 test('Open Image to WebP page', async ({ page }) => {
924   await page.goto('https://www.pixelssuite.com/to-webp');
925
926   await expect(page.getByText('Convert Image')).toBeVisible();
927 });
928
929
930 // 10. Check UI elements
931 test('Image to WebP UI elements', async ({ page }) => {
932   await page.goto('https://www.pixelssuite.com/to-webp');
933
934   await expect(page.getByText('Select files')).toBeVisible();
935   await expect(page.getByText('Preview')).toBeVisible();
936 });
937
938
939 // 11. Upload image and convert
940 test('Upload image for WebP conversion', async ({ page }) => {
941   await page.goto('https://www.pixelssuite.com/to-webp');
942
943   const fileInput = page.locator('input[type="file"]');
944
945   // Upload JPG or PNG file
946   await fileInput.setInputFiles('tests/files/test.jpg');
947
948   await page.waitForTimeout(2000);
949 });
950
951
952 // 12. Screenshot
953 test('Image to WebP screenshot', async ({ page }) => {
954   await page.goto('https://www.pixelssuite.com/to-webp');
955
956   await page.screenshot({ path: 'to-webp.png', fullPage: true });
957 });
958

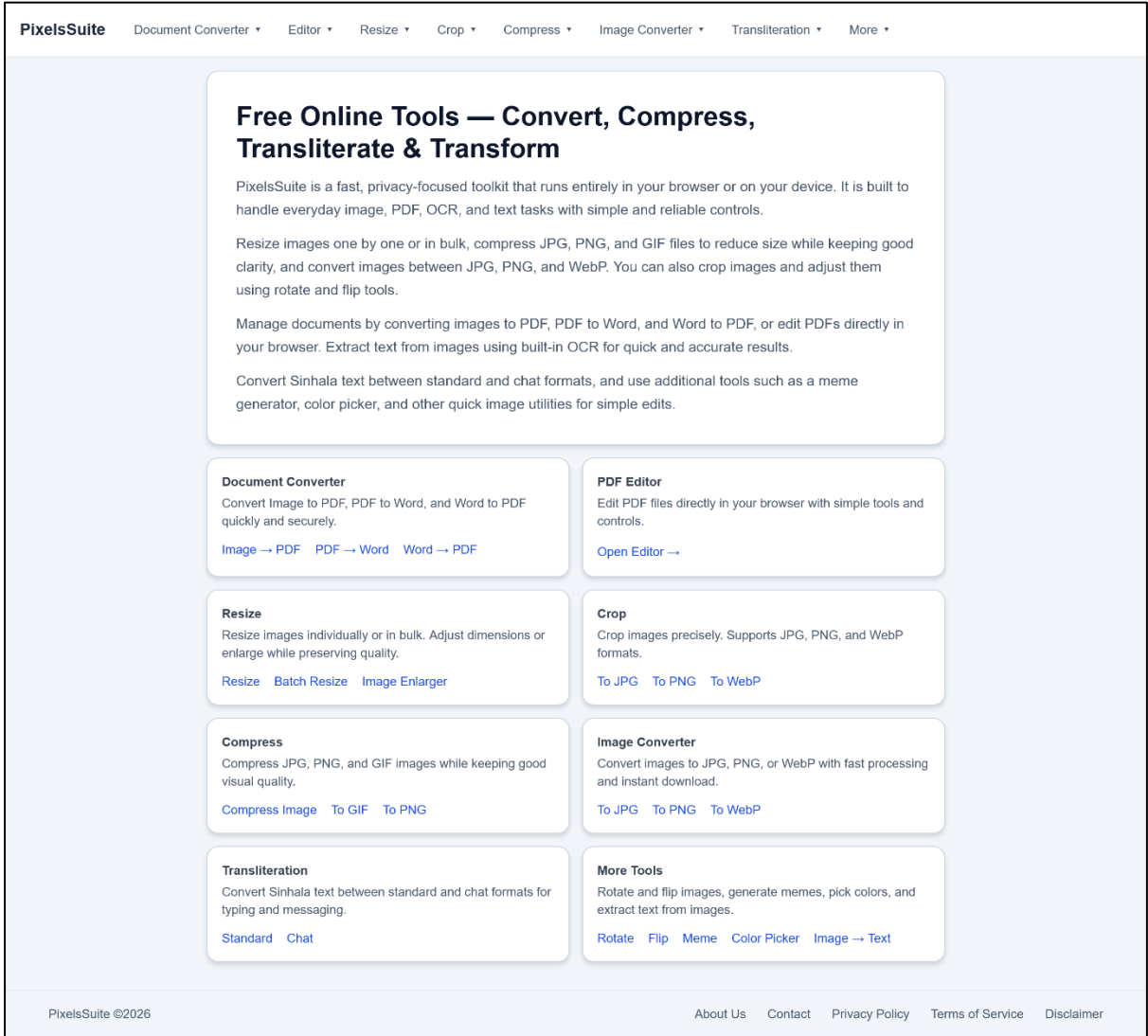
```

Test execution results with screenshots

Test Case ID	Test Case Scenario	Expected Results	Actual Results	Status
ATC_001	Open Homepage	System loads the home page successfully	Homepage loads correctly	Pass
ATC_002	Open Image to PDF page	System loads the image to the PDF page successfully	Image to PDF page loads correctly	Pass
ATC_003	Open Word to PDF page	System loads Word to PDF page successfully	Words to PDF page loads correctly	Pass
ATC_004	Open PDF to Word page	System loads PDF to Word page successfully	PDF to Word page loads correctly	Pass
ATC_005	Open the PDF Editor page	System loads the PDF editor page successfully	PDF Editor page loads correctly	Pass
ATC_006	Open Resize Image page	System loads resize image page successfully	Resize Image page loads correctly	Fail
ATC_007	Open Bulk Resize page	System loads Bulk Resize page successfully	Bulk Resize page loads correctly	Fail
ATC_008	Open the Image Enlarger page	System loads the Image Enlarger page successfully	Image Enlarger page loads correctly	Pass
ATC_009	Open the Crop PNG page	System loads the Crop PNG page successfully	Crop PNG page loads correctly	Pass
ATC_010	Open the Crop JPG page	System loads the Crop JPG page successfully	Crop JPG page loads correctly	Pass
ATC_011	Open Crop WebP page	System loads the Crop Webp page successfully	Crop Webp page loads correctly	Pass
ATC_012	Open Compress Image page	System loads Compress Image page successfully	Compress Image page loads correctly	Pass
ATC_013	Open PNG Compressor page	System loads PNG Compressor page successfully	PNG Compressor page loads correctly	Pass
ATC_014	Open GIF Compressor page	System loads the GIF Compressor page successfully	GIF Compressor page loads correctly	Pass
ATC_015	Open the Image to PNG page	System loads the image to the PNG page successfully	Image to PNG page loads correctly	Fail
ATC_016	Open the Image to JPG page	System loads the Image to the JPG page successfully	Image to JPG page loads correctly	Fail
ATC_017	Open Image to WebP page	System loads the image to the WebP page successfully	Image to WebP page loads correctly	Fail

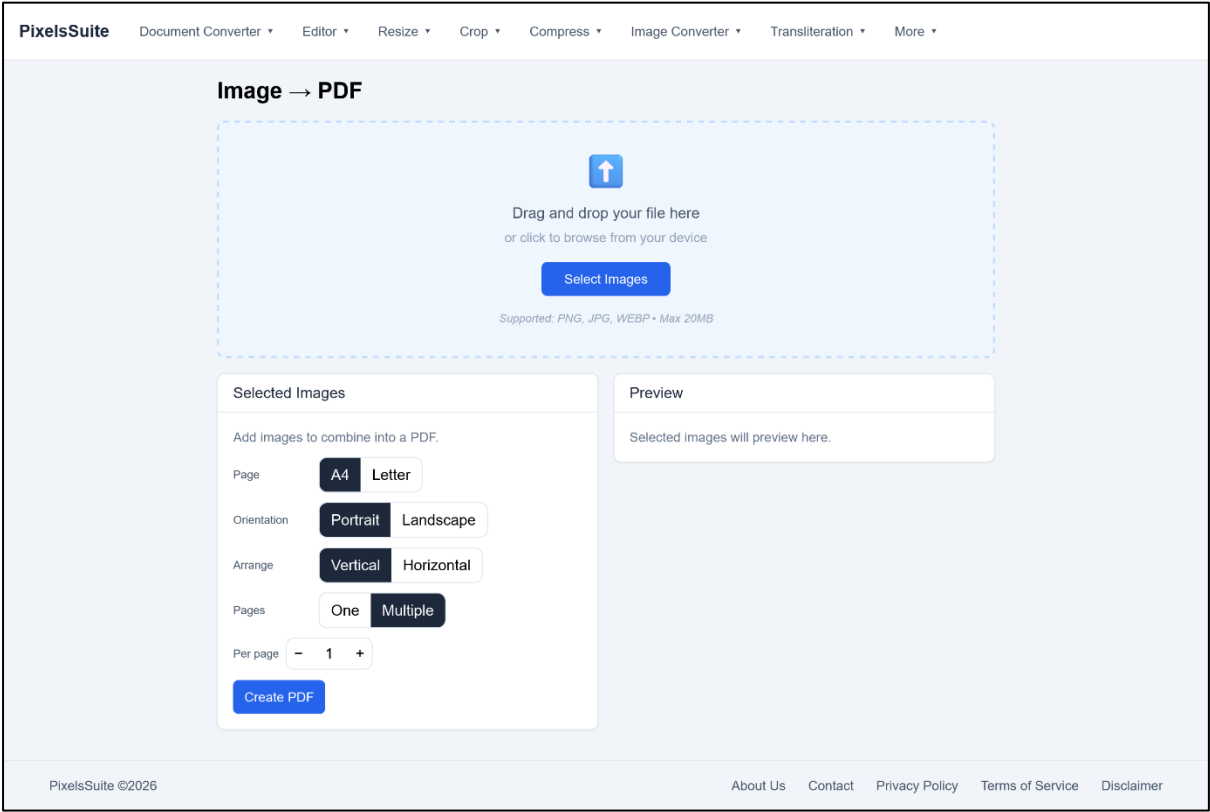
Test Case ID: ATC_001
Test Scenario: Homepage loads correctly
Status: Pass

Screenshot:



Test Case ID: ATC_002
Test Scenario: Open Image to PDF page
Status: Pass

Screenshot:

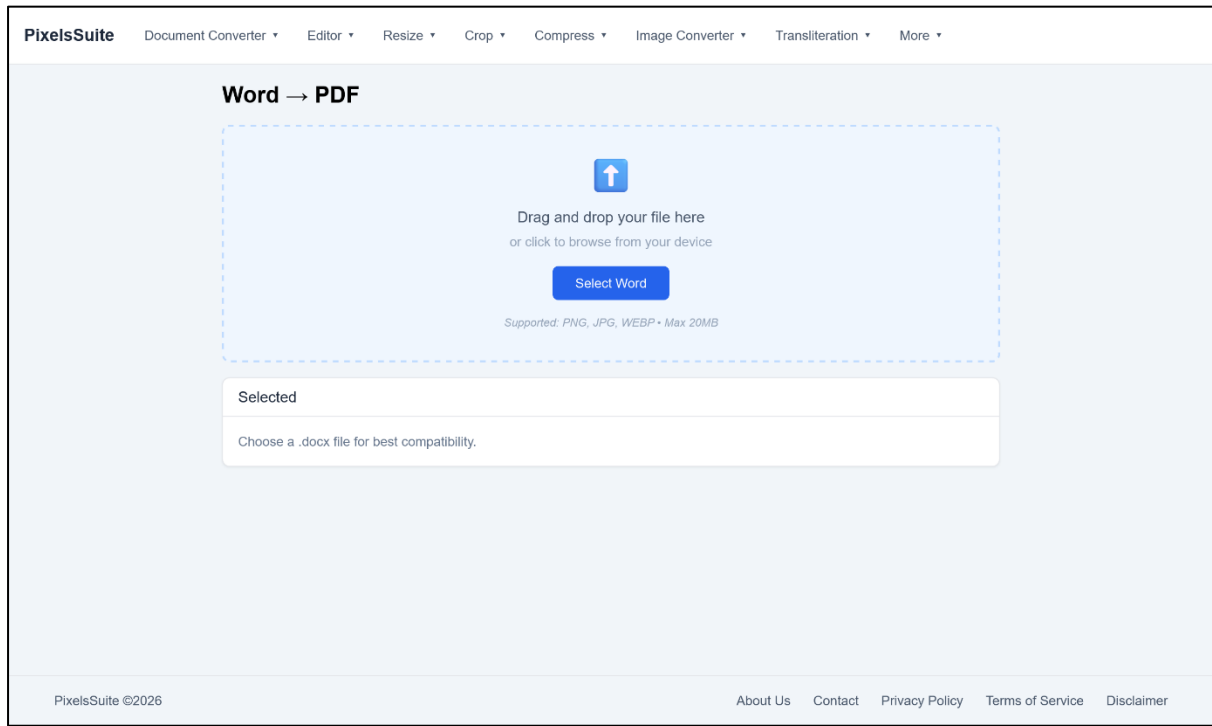


Test Case ID: ATC_003

Test Scenario: Open Word to PDF page

Status: Pass

Screenshot:

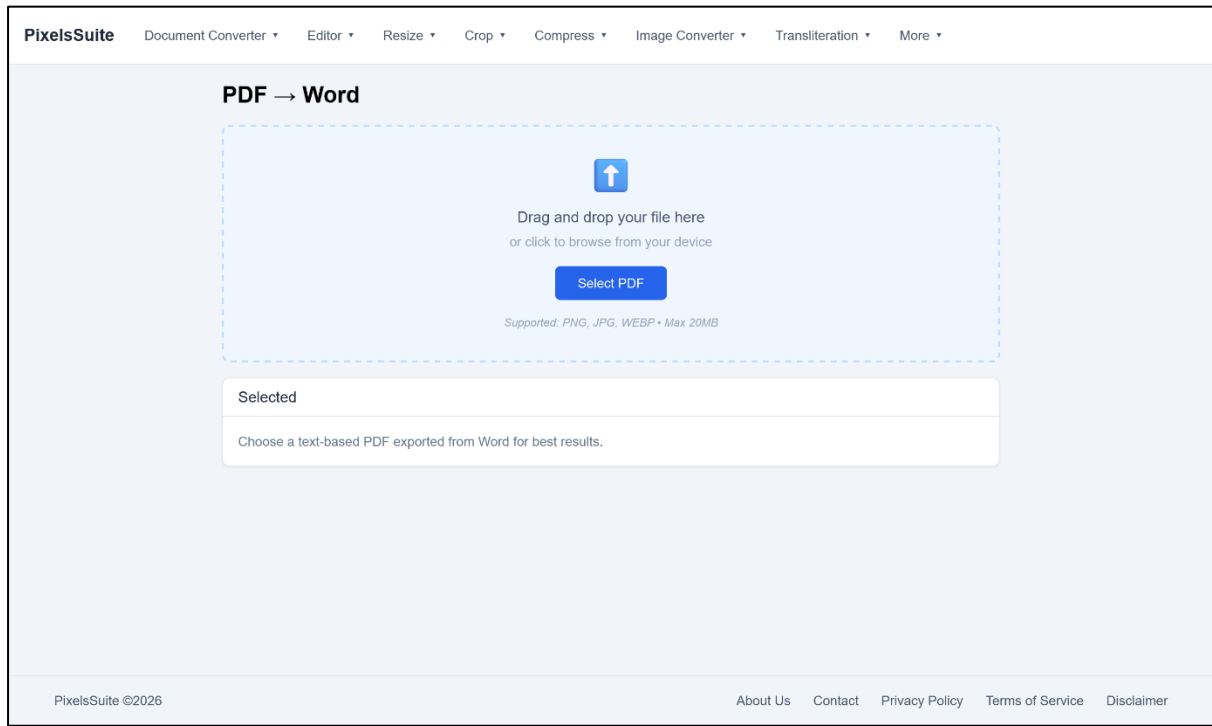


Test Case ID: ATC_004

Test Scenario: Open PDF to Word page

Status: Pass

Screenshot:

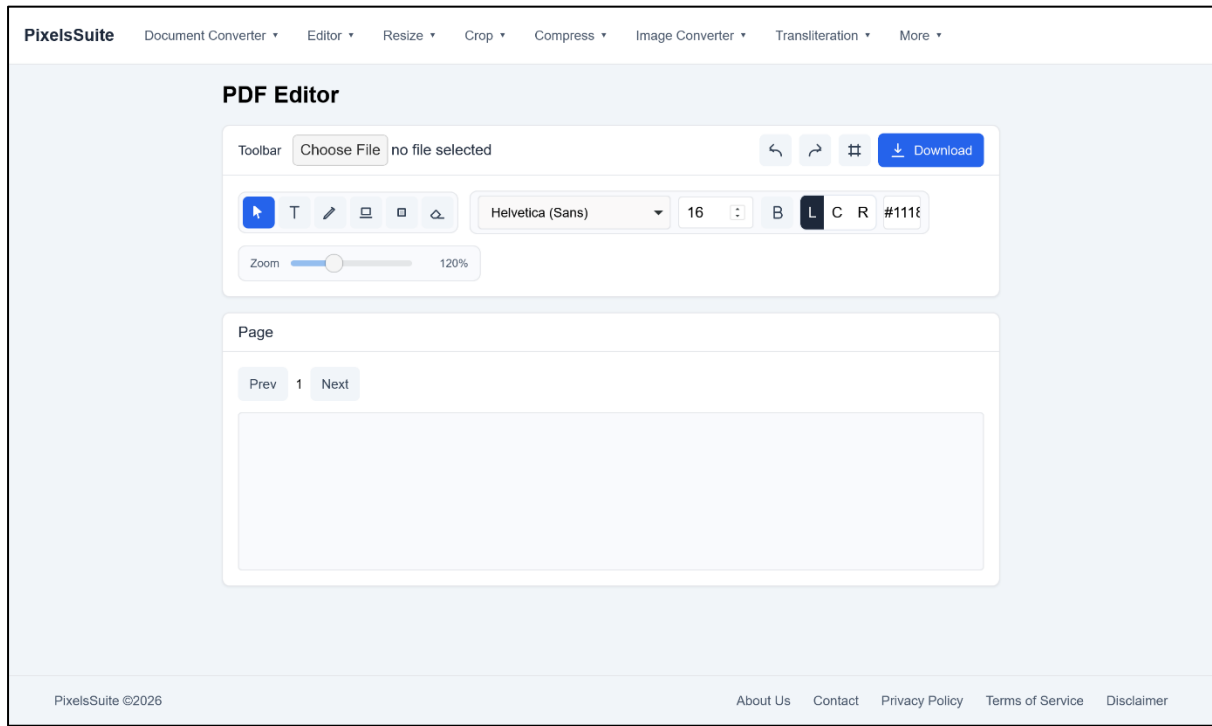


Test Case ID: ATC_005

Test Scenario: Open PDF Editor page

Status: Pass

Screenshot:

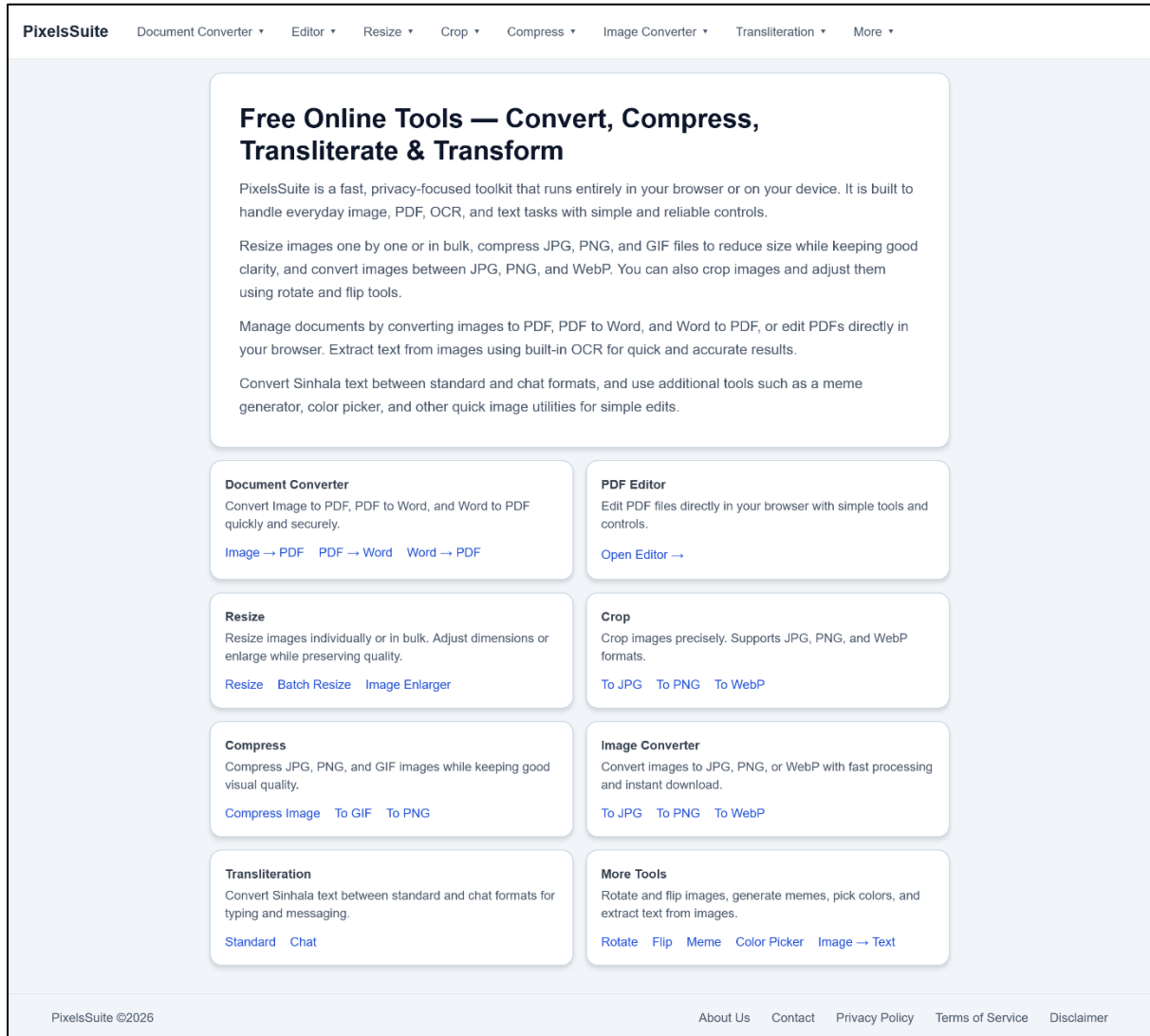


Test Case ID: ATC_006

Test Scenario: Open Resize Image page

Status: Fail

Screenshot:



Test Case ID: ATC_007

Test Scenario: Open Bulk Resize page

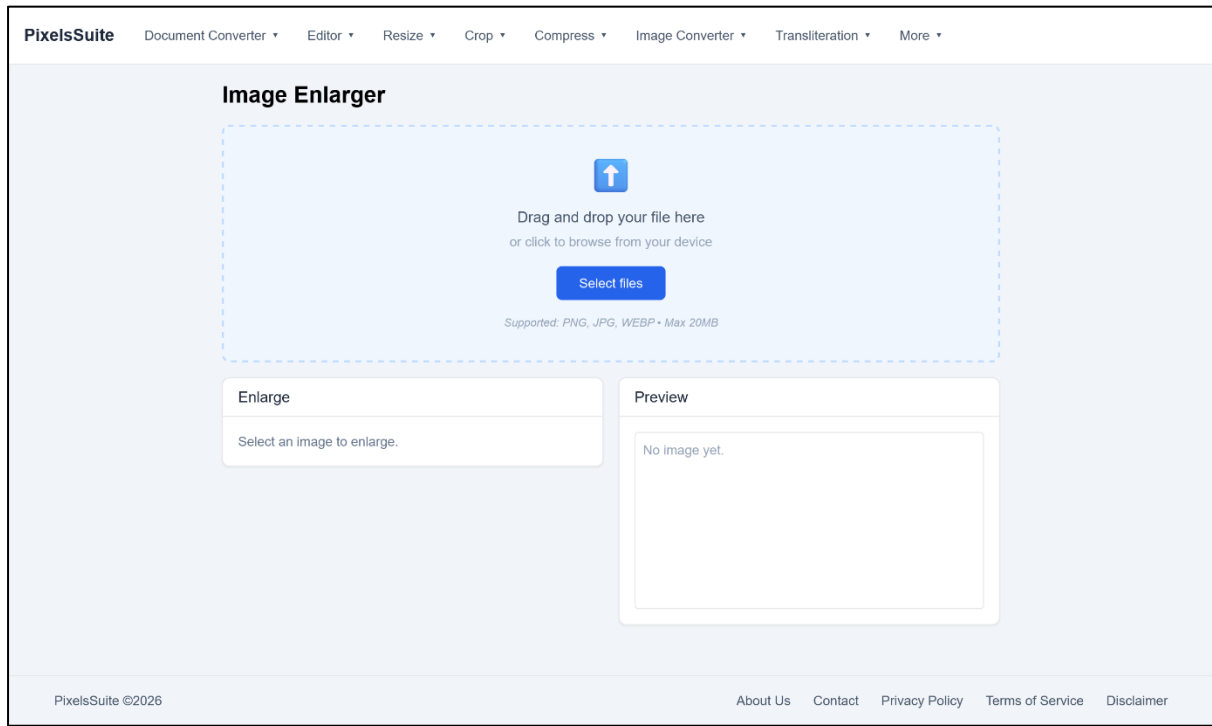
Status: Fail

Test Case ID: ATC_008

Test Scenario: Open Image Enlarger page

Status: Pass

Screenshot:

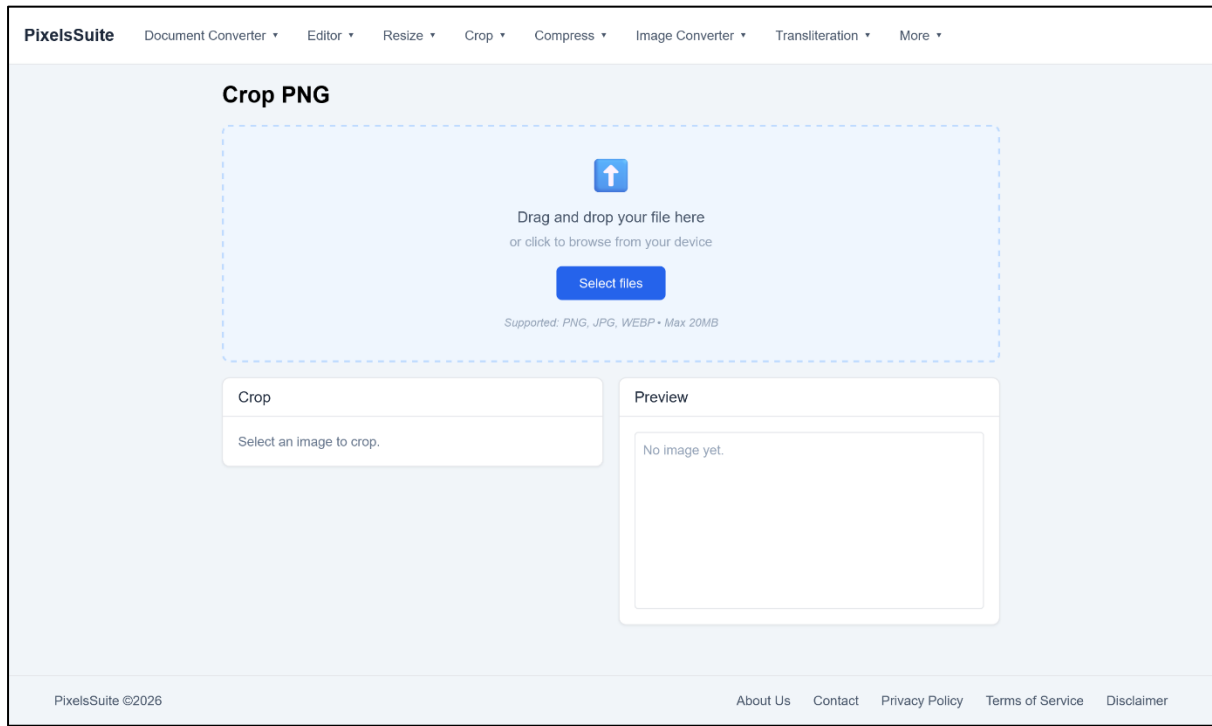


Test Case ID: ATC_009

Test Scenario: Open Crop PNG page

Status: Pass

Screenshot:

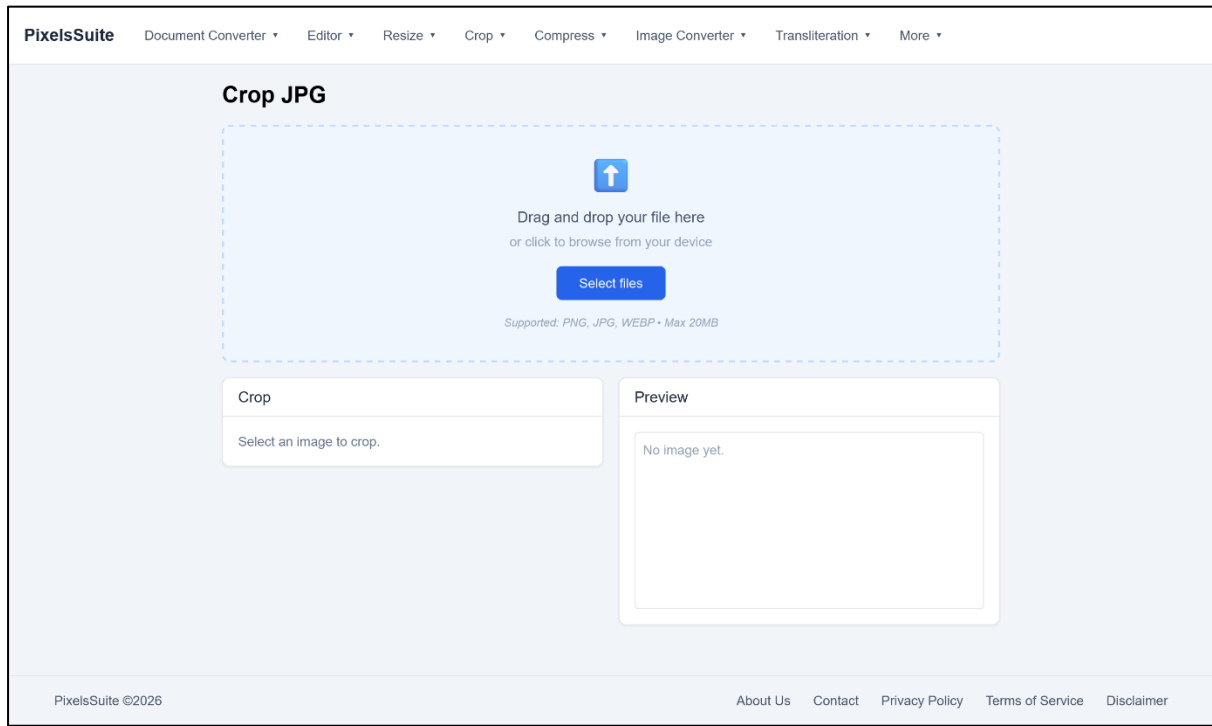


Test Case ID: ATC_010

Test Scenario: Open Crop JPG page

Status: Pass

Screenshot:

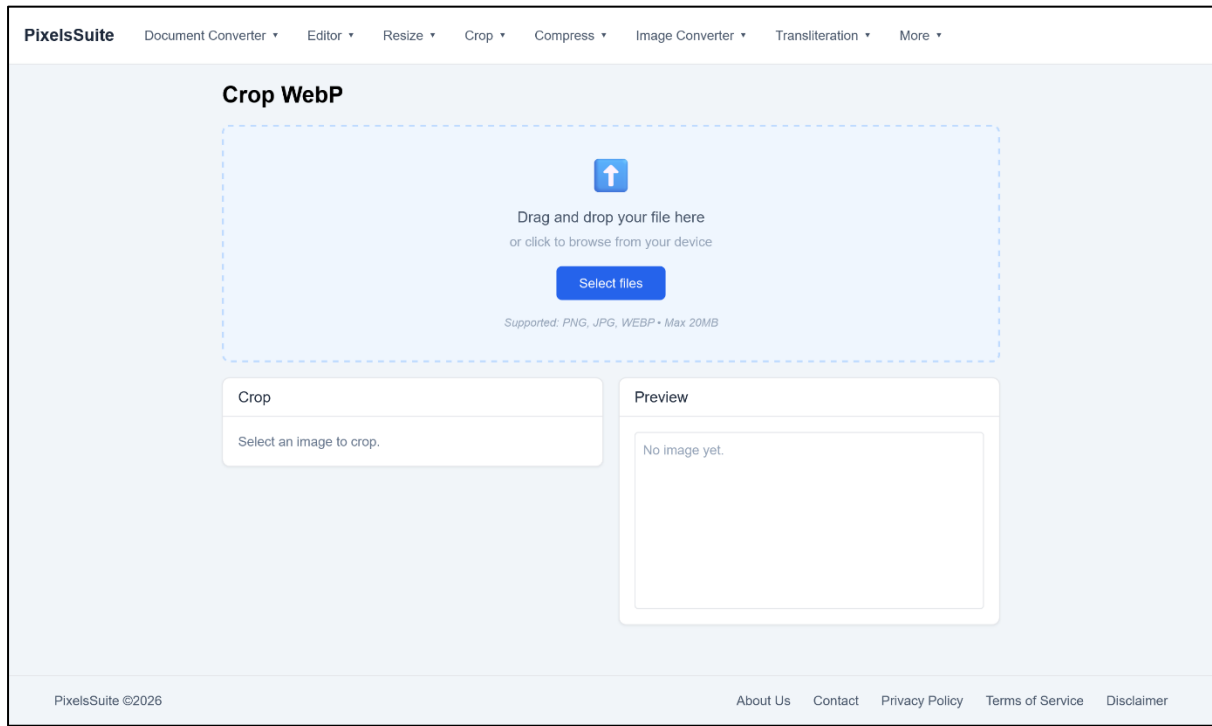


Test Case ID: ATC_011

Test Scenario: Open Crop WebP page

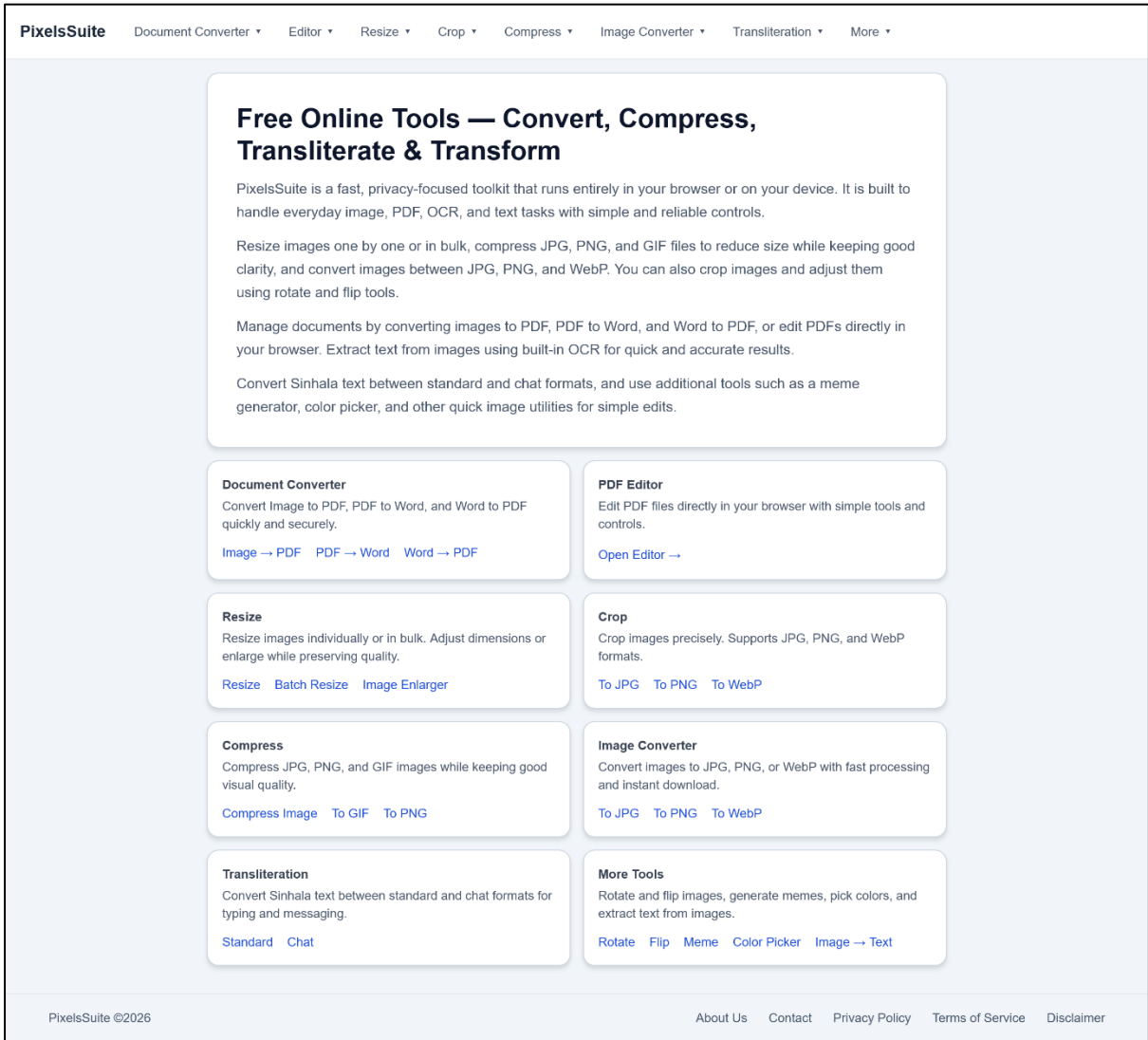
Status: Pass

Screenshot:



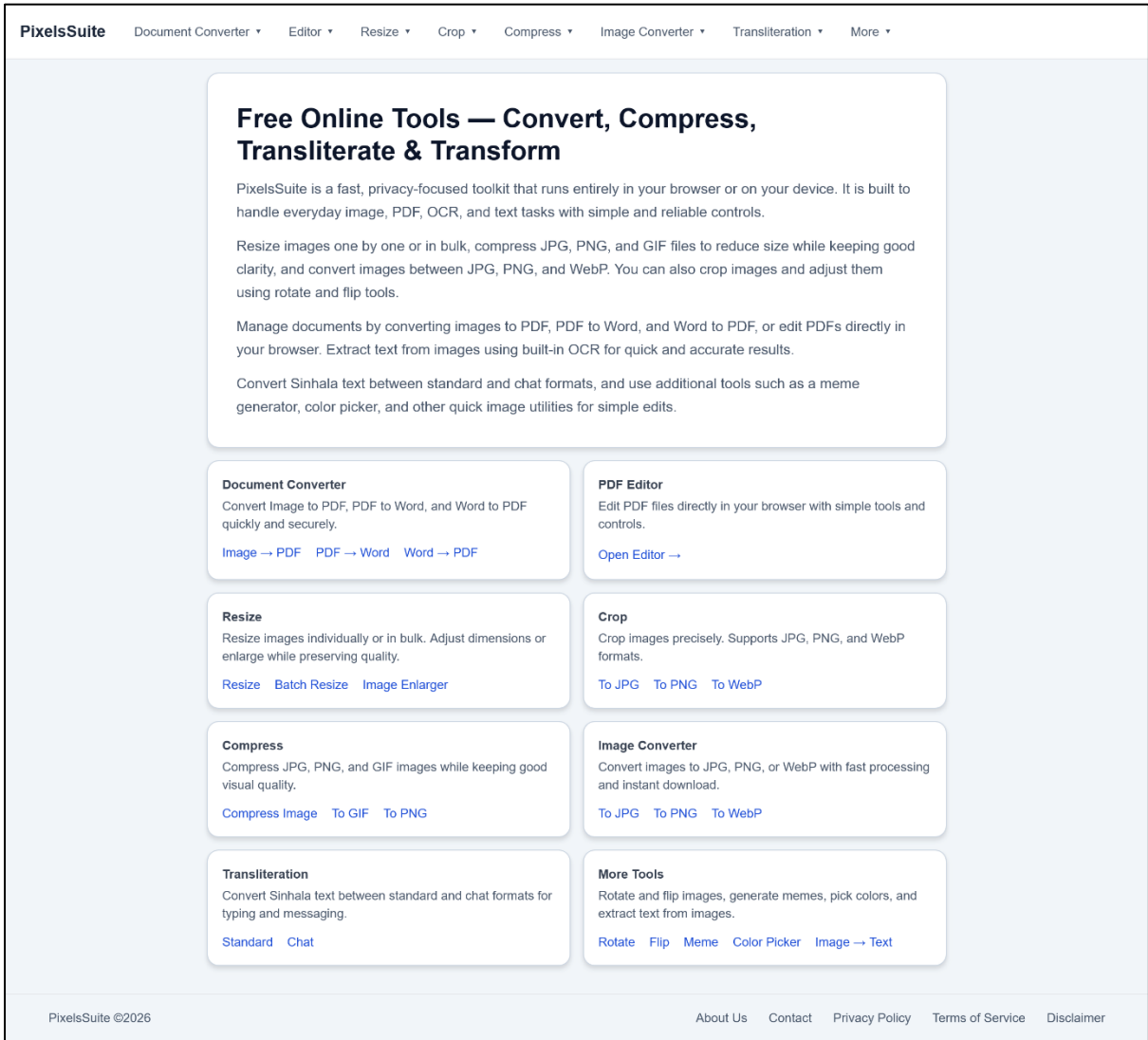
Test Case ID: ATC_012
Test Scenario: Open Compress Image page
Status: Pass

Screenshot:



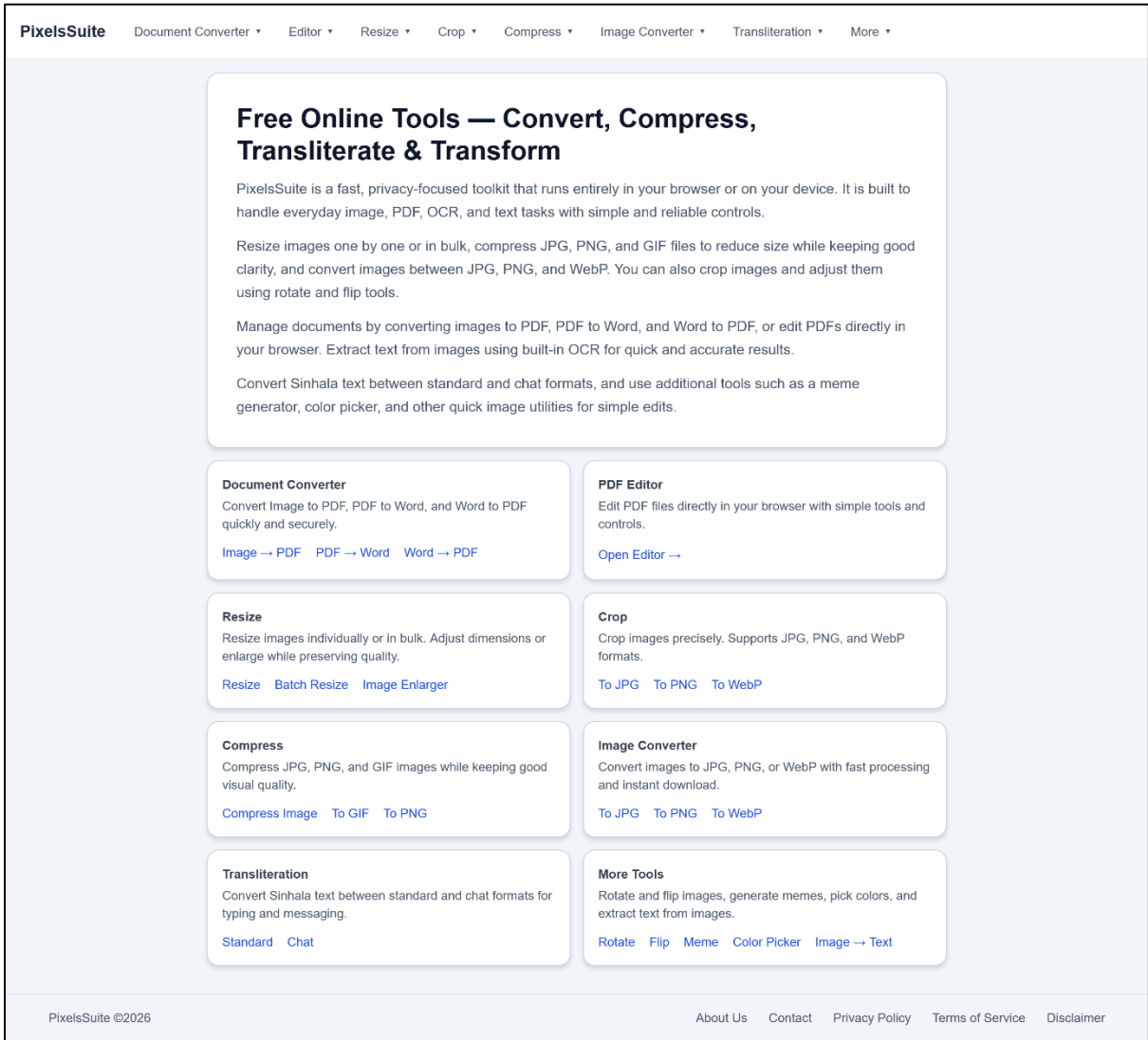
Test Case ID: ATC_013
Test Scenario: Open PNG Compressor page
Status: Pass

Screenshot:



Test Case ID: ATC_014
Test Scenario: Open GIF Compressor page
Status: Fail

Screenshot:

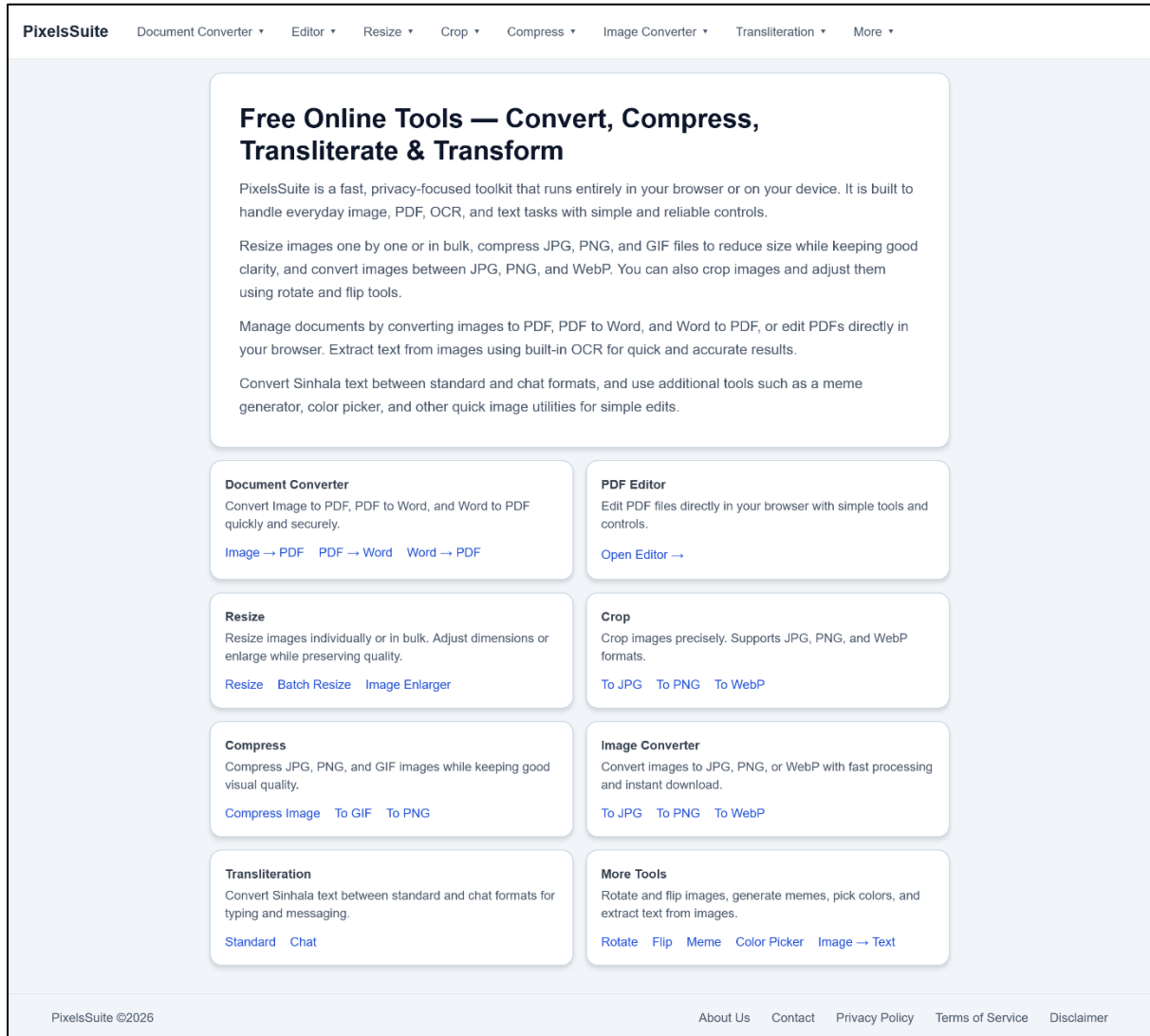


Test Case ID: ATC_015

Test Scenario: Open Image to PNG page

Status: Fail

Screenshot:

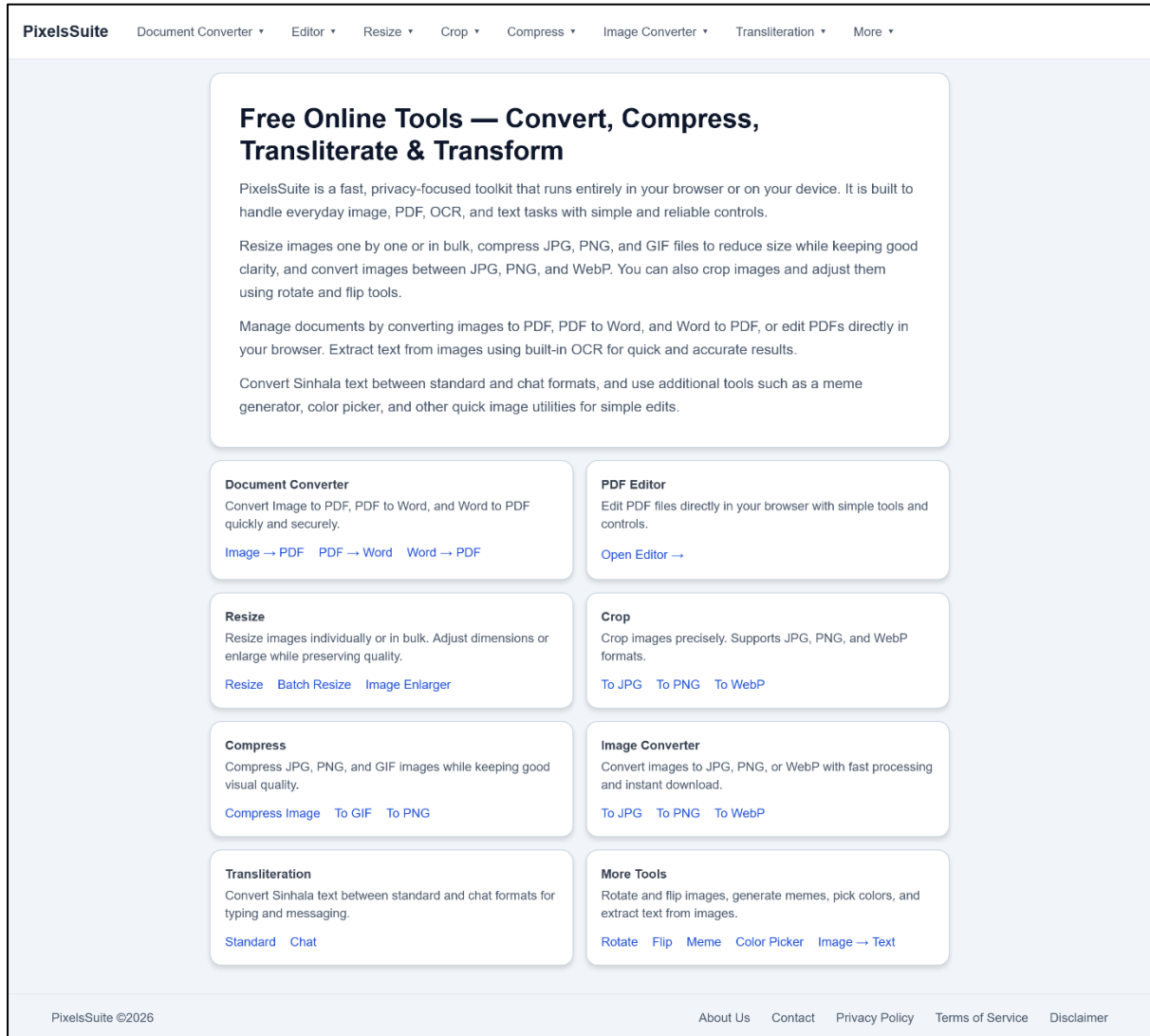


Test Case ID: ATC_016

Test Scenario: Open Image to JPG page

Status: Fail

Screenshot:

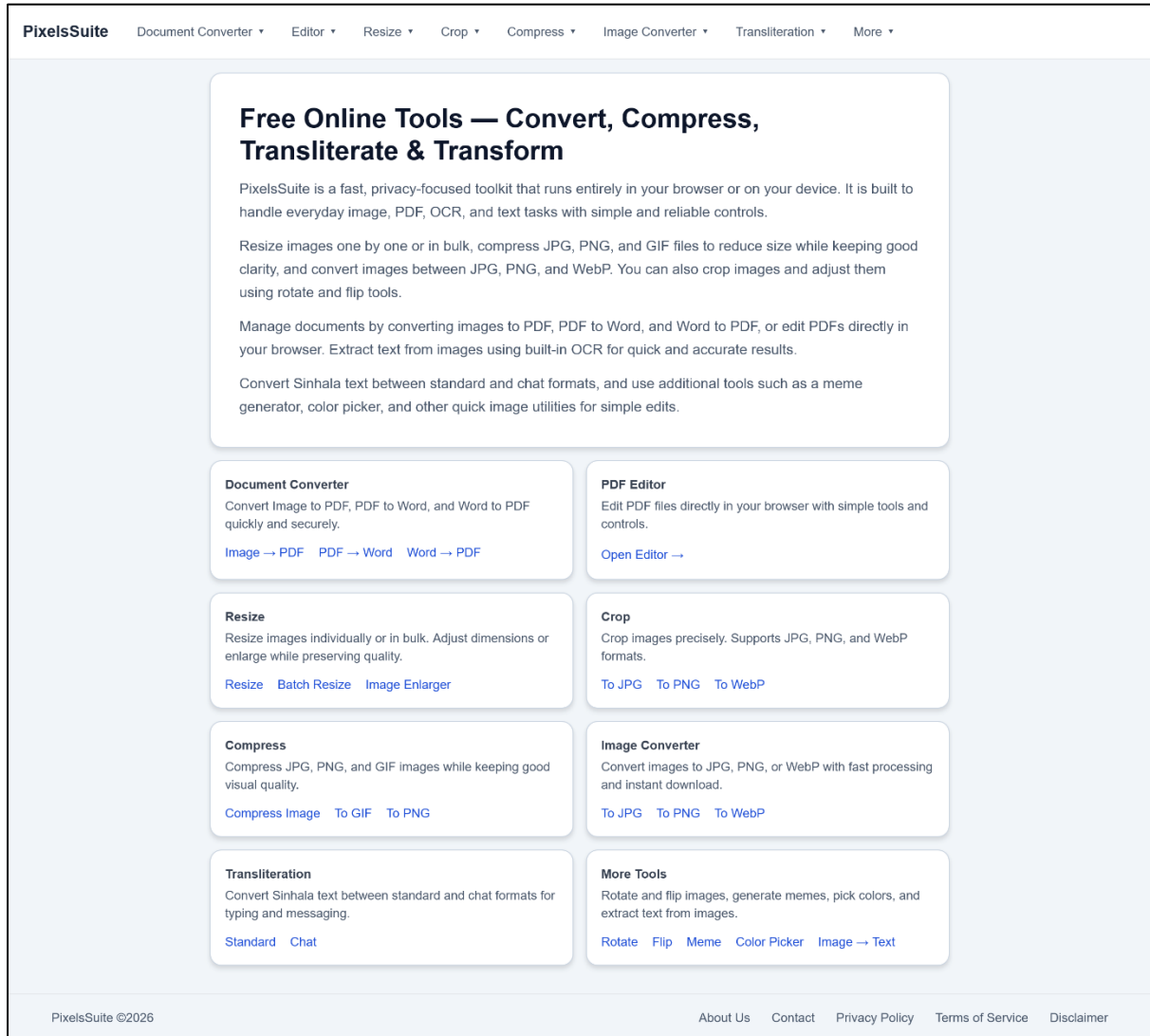


Test Case ID: ATC_017

Test Scenario: Open Image to WebP page

Status: Fail

Screenshot:



Defects found

Search tests

All

258

✓ Passed

141

✗ Failed

117

Flaky

0

Skipped

0

4/18/2026, 2:08:54 PM

Total time: 10.8m

▼

example.spec.js

✗

Document Converter dropdown opens

chromium

4.7s

example.spec.js:14

✗

Navigate to Image → PDF page

chromium

5.2s

example.spec.js:23

✗

Test multiple document converter links

chromium

4.7s

example.spec.js:35

✗

Resize menu navigation

chromium

5.5s

example.spec.js:53

✗

Crop menu navigation

chromium

4.5s

example.spec.js:65

✗

Compress menu navigation

chromium

2.1s

example.spec.js:77

✗

Create PDF without uploading file

chromium

30.1s

example.spec.js:208

✗

Word to PDF without file (edge case)

chromium

30.1s

example.spec.js:250

✗

PDF to Word without file (edge case)

chromium

30.1s

example.spec.js:291

✗

PDF Editor UI elements visible

chromium

7.5s

example.spec.js:319

✗

Zoom slider interaction

chromium

30.1s

example.spec.js:374

✗

Page navigation buttons

chromium

30.1s

example.spec.js:391

✗ Download without uploading PDF (edge case) chromium	example.spec.js:405	30.4s
✗ Open Resize Image page chromium	example.spec.js:431	1.8s
✗ Resize Image UI elements chromium	example.spec.js:440	6.4s
✗ Upload image in Resize page chromium	example.spec.js:450	30.1s
✗ Open Bulk Resize page chromium	example.spec.js:476	6.6s
✗ Bulk Resize UI elements chromium	example.spec.js:484	6.6s
✗ Upload multiple images chromium	example.spec.js:495	30.3s
✗ Enter dimensions and process images chromium	example.spec.js:511	30.2s
✗ Bulk Resize without upload (edge case) chromium	example.spec.js:529	30.3s
✗ Compress Image UI elements chromium	example.spec.js:719	6.9s
✗ Upload image for compression chromium	example.spec.js:729	30.1s
✗ Open PNG Compressor page chromium	example.spec.js:754	7.2s
✗ PNG Compressor UI elements chromium	example.spec.js:762	6.6s
✗ Upload PNG for compression chromium	example.spec.js:771	30.2s

✗ Open GIF Compressor page	chromium	7.9s
example.spec.js:796		
✗ GIF Compressor UI elements	chromium	7.1s
example.spec.js:804		
✗ Upload GIF for compression	chromium	30.2s
example.spec.js:813		
✗ Open Image to PNG page	chromium	2.2s
example.spec.js:837		
✗ Image to PNG UI elements	chromium	7.6s
example.spec.js:846		
✗ Upload image for PNG conversion	chromium	30.2s
example.spec.js:856		
✗ Open Image to JPG page	chromium	2.0s
example.spec.js:881		
✗ Image to JPG UI elements	chromium	7.5s
example.spec.js:889		
✗ Upload image for JPG conversion	chromium	30.4s
example.spec.js:898		
✗ Open Image to WebP page	chromium	2.2s
example.spec.js:923		
✗ Image to WebP UI elements	chromium	7.1s
example.spec.js:931		
✗ Upload image for WebP conversion	chromium	30.1s
example.spec.js:940		
✗ Document Converter dropdown opens	firefox	16.3s
example.spec.js:14		
✗ Navigate to Image → PDF page	firefox	11.9s
example.spec.js:23		

✗ Test multiple document converter links firefox	11.9s
example.spec.js:35	
✗ Resize menu navigation firefox	6.4s
example.spec.js:53	
✗ Crop menu navigation firefox	25.0s
example.spec.js:65	
✗ Compress menu navigation firefox	16.5s
example.spec.js:77	
✗ Button click works firefox	30.7s
example.spec.js:103	
✗ Empty input validation firefox	31.6s
example.spec.js:117	
✗ Create PDF after upload firefox	30.5s
example.spec.js:192	
✗ Create PDF without uploading file firefox	30.7s
example.spec.js:208	
✗ Word to PDF without file (edge case) firefox	31.8s
example.spec.js:250	
✗ PDF to Word without file (edge case) firefox	30.9s
example.spec.js:291	
✗ PDF Editor UI elements visible firefox	9.7s
example.spec.js:319	
✗ Zoom slider interaction firefox	31.0s
example.spec.js:374	
✗ Page navigation buttons firefox	30.6s
example.spec.js:391	
✗ Download without uploading PDF (edge case) firefox	30.4s
example.spec.js:405	

✗ Open Resize Image page firefox	4.9s
example.spec.js:431	
✗ Resize Image UI elements firefox	17.5s
example.spec.js:440	
✗ Upload image in Resize page firefox	33.2s
example.spec.js:450	
✗ Open Bulk Resize page firefox	8.9s
example.spec.js:476	
✗ Bulk Resize UI elements firefox	24.6s
example.spec.js:484	
✗ Upload multiple images firefox	30.3s
example.spec.js:495	
✗ Enter dimensions and process images firefox	31.8s
example.spec.js:511	
✗ Bulk Resize without upload (edge case) firefox	30.7s
example.spec.js:529	
✗ Compress Image UI elements firefox	10.0s
example.spec.js:719	
✗ Upload image for compression firefox	30.4s
example.spec.js:729	
✗ Open PNG Compressor page firefox	10.5s
example.spec.js:754	
✗ PNG Compressor UI elements firefox	11.1s
example.spec.js:762	
✗ Upload PNG for compression firefox	31.2s
example.spec.js:771	
✗ Open GIF Compressor page firefox	8.1s
example.spec.js:796	

✗ GIF Compressor UI elements	firefox	15.8s
example.spec.js:804		
✗ Upload GIF for compression	firefox	31.8s
example.spec.js:813		
✗ Open Image to PNG page	firefox	11.5s
example.spec.js:837		
✗ Image to PNG UI elements	firefox	19.4s
example.spec.js:846		
✗ Upload image for PNG conversion	firefox	32.5s
example.spec.js:856		
✗ Open Image to JPG page	firefox	11.8s
example.spec.js:881		
✗ Image to JPG UI elements	firefox	17.0s
example.spec.js:889		
✗ Upload image for JPG conversion	firefox	31.5s
example.spec.js:898		
✗ Open Image to WebP page	firefox	17.9s
example.spec.js:923		
✗ Image to WebP UI elements	firefox	15.9s
example.spec.js:931		
✗ Upload image for WebP conversion	firefox	30.2s
example.spec.js:940		
✗ Document Converter dropdown opens	webkit	4.6s
example.spec.js:14		
✗ Navigate to Image → PDF page	webkit	4.7s
example.spec.js:23		
✗ Test multiple document converter links	webkit	4.1s
example.spec.js:35		
✗ Resize menu navigation	webkit	4.9s
example.spec.js:53		